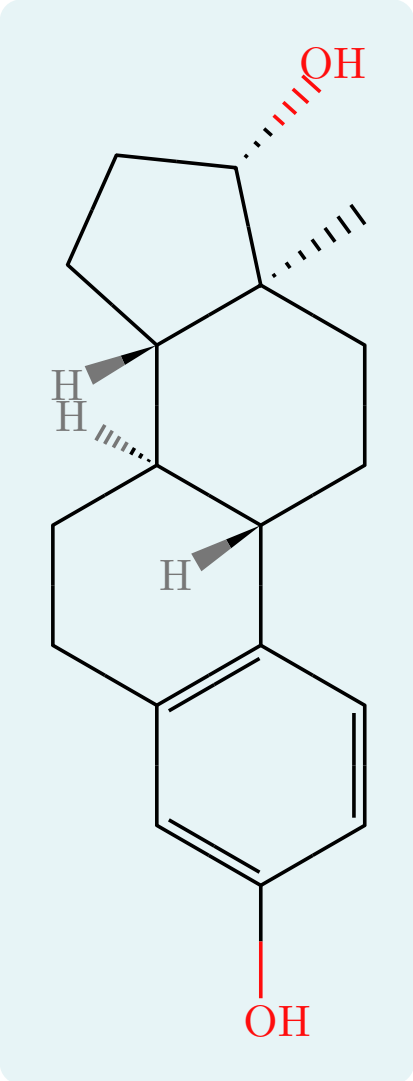
typed-smiles

Render SMILES strings as 2D molecular diagrams

User Guide



Version 0.13.0

Contents

1 Introduction . . . 3
2 Getting started . . . 4
2.1 Installation and import . . . 4
2.2 Your first molecule . . . 4
2.3 Named molecules . . . 5
3 The `smiles()` function . . . 7
3.1 Argument reference . . . 7
3.2 `smiles-str` . . . 10
3.3 `style` . . . 11
3.4 `scale` . . . 12
3.5 `bond-length` . . . 12
3.6 `font-size` . . . 12
3.7 `font` . . . 13
3.8 `bond-stroke` . . . 13
3.9 `color` . . . 14
3.10 `fg` and `theme` . . . 14
3.11 `rotation` . . . 15
3.12 `mirror` . . . 16
3.13 `show-h` . . . 17
4 Molecular formula: `mol-formula()` . . . 19
4.1 `atom-annotations` . . . 19
4.2 `opacity` . . . 20
4.3 `bond-customizations` . . . 21
4.4 `lone-pairs` . . . 21
5 Atoms and charges . . . 22
5.1 Aromatic rings . . . 22
5.2 Charges . . . 25
5.3 Salts and disconnected structures . . . 26
5.4 Atom maps . . . 26
6 Hydrogens . . . 27
6.1 Default display . . . 27
6.2 Explicit bracket hydrogens . . . 27
6.3 Isotopes . . . 28
6.4 Label orientation . . . 28
7 Stereochemistry . . . 29
7.1 Tetrahedral centers: `@` and `@@` . . . 29
7.2 Double bonds: `/` and `\` . . . 30
7.3 Square-planar centers: `@SP1`, `@SP2`, `@SP3` . . . 31
7.4 Stereochemistry the drawing cannot show . . . 32
7.5 Manual wedges: `!w` and `!h` . . . 32
7.6 Wavy and dashed bonds: `!s` and `!d` . . . 33
7.7 Curling an acyclic chain: `!c` . . . 34
8 Colors . . . 37
8.1 Jmol CPK palette . . . 37
8.2 Custom colors (`atom-colors`) . . . 37
8.3 Label colors (`{label|style}`) . . . 39
9 Abbreviated groups . . . 41
9.1 Plain labels . . . 41

9.2 Colored labels . . . 41
9.3 Scripted labels . . . 42
9.4 Lone pairs on labels . . . 43
9.5 Manual label offsets . . . 44
9.6 Automatic abbreviations . . . 44
10 Chemical formulas: `ce()` . . . 47
10.1 Fonts and size . . . 47
10.2 Charge placement and style . . . 47
11 Molecular weight: `mol-weight()` . . . 48
12 Inline molecules: `smiles-inline()` . . . 49
13 CeTZ integration: `smiles-cetz()` . . . 50
14 Reaction schemes . . . 52
14.1 `mol()` . . . 52
14.2 `rxn-arrow()` . . . 52
14.3 `reaction()` . . . 55
15 Reaction mechanisms . . . 60
15.1 Referencing atoms . . . 60
15.2 Automatic substructure highlighting . . . 61
15.3 `arrow()` and `highlight()` . . . 65
15.4 A mechanism across species . . . 68
15.5 Per-molecule scale in mechanisms . . . 68
15.6 Referencing molecules above an arrow . . . 69
15.7 `brackets()` . . . 69
16 Catalytic cycles: `cycle()` . . . 70
17 Molecule series . . . 73
17.1 Aligning a series: `align-molecules()` . . . 73
17.2 Compound grids: `molecule-grid()` . . . 75
18 Project-wide defaults . . . 77
19 Limitations . . . 78
20 Molecule library . . . 79
21 Quick reference . . . 84
21.1 Syntax . . . 84
21.2 `smiles()` options . . . 84
21.3 `reaction()` options . . . 85
21.4 `rxn-arrow()` options . . . 86
21.5 Data helpers . . . 86
21.6 Mechanism helpers . . . 87
21.7 Molecule series . . . 88
21.8 Label color names . . . 88

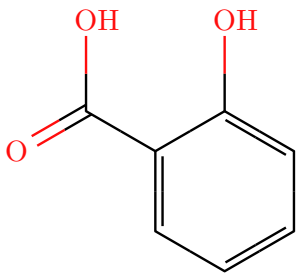
1 Introduction

typed-smiles renders SMILES strings as 2D skeletal molecular diagrams inside Typst documents. Use it for molecule drawings, atom labels, charges, hydrogens, stereochemical bonds, formulas, and reaction schemes.

This guide documents every function, argument, and package-specific syntax extension. Each feature comes with a runnable example: the source is on the left, its rendered output on the right.

```typst
#smiles("OC1=CC=CC=C1C(=O)O")
```

Typst



2 Getting started

2.1 Installation and import

Import the package from the Typst preview namespace. A wildcard import gives you every public symbol:

```
#import "@preview/typed-smiles:0.13.0": *
```
Typst

Or import only what you need:

```
#import "@preview/typed-smiles:0.13.0": smiles, ce, mol, rxn-arrow, reaction
```
Typst

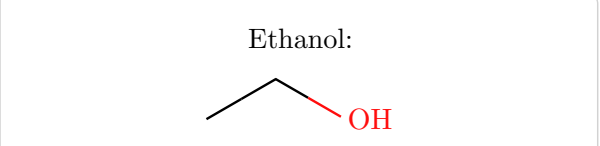
The package exports these main symbols:

Symbol	Purpose
`smiles`	Render a SMILES string as a molecule (the main function).
`ce`	Chemical formulas and equations, re-exported from `chemformula`.
`mol`	Wrap a molecule with an optional caption.
`rxn-arrow`	A reaction arrow with conditions above and below.
`reaction`	Lay out a multi-step reaction scheme.
`atom`, `bond`, `lp`, `species`	References for arrows and highlights.
`arrow`, `highlight`	Mechanism arrows and shaded atom/bond regions.
`brackets`	Draw square brackets with optional corner marks.
`mol-formula`	Compute and render a molecular formula from a SMILES string.
`mol-weight`	Compute molecular weight from a SMILES string.
`molecules`	Bundled SMILES strings for common molecules, looked up by name.

2.2 Your first molecule

Call `smiles` with a SMILES string.

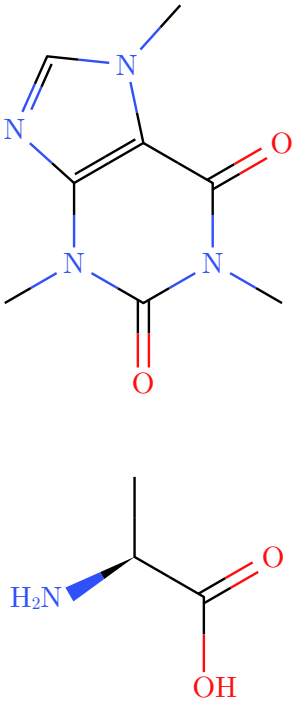
```
Ethanol: #smiles("CCO")
```
Typst



2.3 Named molecules

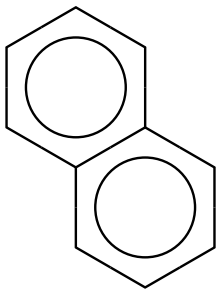
`molecules` is a dictionary of SMILES strings for over a hundred common molecules, bundled with the package so it works offline. Write `molecules.` followed by a name; the editor suggests the available names as you type.

```typst
#smiles(molecules.caffeine) \
#smiles(molecules.alanine)
```

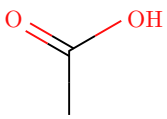


Each entry is an ordinary SMILES string, so it works with every function and argument that accepts one. Names are lowercase and hyphenated; use `molecules.at("name")` when the name comes from a variable.

```typst
#smiles(molecules.naphthalene,
aromatic: "circle") \
#mol-formula(molecules.aspirin),
#calc.round(mol-weight(molecules.aspirin),
digits: 2) g/mol \
#smiles(molecules.at("acetic-acid"),
scale: 0.8)
```



$C_9H_8O_4$, 180.16 g/mol



Note. Stereocenters encode the natural or commonly sold isomer: L amino acids, D sugars (glucose as the pyranose, ribose as the furanose), and, for example, (−)-menthol, (S)-naproxen, and (S)-

> nicotine. Ibuprofen is racemic. A misspelled name stops compilation with a `dictionary does not contain key` error at the call site. See Section 20 for every entry.

3 The `smiles()` function

`smiles` is the main function. Its full signature is:

```typst
#smiles(
  smiles-str,
  style: "default",
  scale: 1.0,
  bond-length: none,
  font-size: none,
  font: auto,
  bond-stroke: none,
  color: auto,
  fg: auto,
  theme: auto,
  rotation: 0deg,
  mirror: none,
  show-h: (),
  aromatic: "kekule",
  atom-annotations: (),
  opacity: 100%,
  bond-customizations: (),
  lone-pairs: none,
  atom-colors: (:),
  show-indices: false,
  abbreviate: none,
  highlight-smarts: (),
  highlight-groups: (),
  highlight-colors: auto,
  highlight-unmatched: "error",
  highlight-overlap: "merge",
  show-maps: false,
  undepicted-stereo: "error",
  ..annotations
)
```

3.1 Argument reference

Argument	Type	Default	Description
`smiles-str`	`str`	(required)	The SMILES string.
`style`	`str`	`"default"`	Journal style preset (`"acs"`, `"rsc"`, `"nature"`, `"wiley"`) filling in bond length, label size, stroke, and font. Explicit arguments win; `"default"` applies nothing.

`scale`	`float`	`1.0`	Balanced scale for bond length, label size, and stroke at once.
`bond-length`	`float` / `none`	`none`	Bond length factor (`1.0` is 30 pt). Overrides `scale` for length.
`font-size`	`length` / `none`	`none`	Atom-label font size. Overrides `scale` for labels.
`font`	`auto` / `str` / `array`	`auto`	Font family for atom labels; `auto` is "New Computer Modern", or the preset's font when `style` is set.
`bond-stroke`	`length` / `none`	`none`	Bond stroke width. Overrides `scale` for strokes.
`color`	`auto` / `bool`	`auto`	Apply Jmol CPK atom colors. `auto` is colored for `"default"` and monochrome for journal presets.
`fg`	`auto` / `color`	`auto`	Foreground for bonds and carbon labels; `auto` inherits the surrounding text color.
`theme`	`auto` / `"light"` / `"dark"`	`auto`	CPK palette variant; `auto` picks "dark" when `fg` is light.
`rotation`	`angle`	`0deg`	Rotate the molecule; labels stay upright.
`mirror`	`none` / `"horizontal"` / `"vertical"`	`none`	Optional horizontal or vertical page-axis reflection.
`show-h`	`"skeleton"` / `"all"` / `int` / `array`	`()`	Implicit hydrogens to label beyond the default heteroatom hydrogens. Use `"all"` for compact labels or `"skeleton"` for separate H atoms and bonds.
`aromatic`	`"kekule"` / `"circle"`	`"kekule"`	Depict lowercase-aromatic rings with alternating double bonds or with an inscribed circle.
`charge-style`	`"plain"` / `"circled"`	`"plain"`	Write formal charges as a bare sign or with the sign inside a circle. See Section 5.2.

`atom-annotations`	`array`	`()`	Small gray side labels as `(index, content)` or `(index, content, offset)` tuple entries.
`opacity`	`ratio` / `float`	`100%`	Fade the whole drawing — bonds, labels, charges, lone pairs — e.g. for ghost molecules.
`bond-customizations`	`array`	`()`	Per-bond style overrides as `(bond(i, j), (..options..))` pairs; options are `color`, `stroke`, and `opacity`.
`lone-pairs`	`none` / `"dots"` / `"lines"`	`none`	Draw optional non-bonding electron pairs on skeletal atom labels.
`atom-colors`	`dictionary`	`(:)`	Color overrides for elements and labels. Element-symbol keys (e.g. `O: red`) override CPK colors; brace-quoted label keys (e.g. `"{PPh3}": purple`) override a specific abbreviated group. See Section 8.
`abbreviate`	`none` / `"all"` / `str` / `array`	`none`	Draw catalogue terminal groups (`"tBu"`, `"CO2Et"`, `"CO2Me"`, `"OAc"`, `"NHAc"`, `"SO3H"`, `"CF3"`, `"NO2"`, `"CO2H"`, `"CO2-"`, `"CN"`, `"CHO"`, `"OEt"`, `"OMe"`, `"Ac"`) as labels. See Section 9.6.
`show-maps`	`bool`	`false`	Label atoms carrying an atom map such as `[CH3:7]` with `:7`. See Section 5.4.
`undepicted-stereo`	`"error"` / `"omit"`	`"error"`	Report written stereochemistry the drawing cannot show, or draw the structure without it. See Section 7.4.
`show-indices`	`bool`	`false`	Show writing-order atom indices for references.
`highlight-smarts`	`str` / `dictionary` / `array`	`()`	SMARTS patterns to shade; see Section 15.
`highlight-groups`	`str` / `dictionary` / `array`	`()`	Named functional groups to shade; see Section 15.

`highlight-colors`	`auto` / `array`	`auto`	Palette for substructure highlights.
`highlight-unmatched`	`"error"` / `"ignore"`	`"error"`	Policy for a pattern or group absent from the molecule.
`highlight-overlap`	`"merge"` / `"stack"`	`"merge"`	Merge overlapping highlight pieces by color or paint each piece separately.
`..annotations`	`arrow()` / `highlight()`	—	Positional annotations on this molecule.

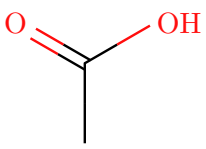
Note. `scale` sizes everything together. Use `bond-length`, `font-size`, and `bond-stroke` to override one dimension on its own; each defaults to `none`, meaning it follows `scale`.

3.2 `smiles-str`

The positional argument is the SMILES string.

```
1  #smiles("CC(=O)O")
```

Typst



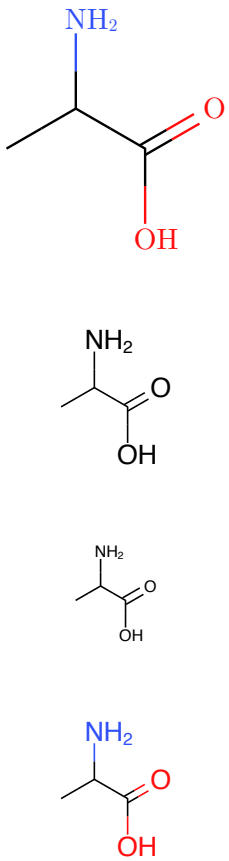
Note. Invalid SMILES produce an editor-visible error that names the character position and the correction, for example `ring closure 1 at character 3 closes on the atom that opened it` for `C11`. Besides syntax errors, the following are rejected: a ring closure that bonds an atom to itself or repeats an existing bond (`C12C12`), different bond symbols on the two ends of one ring closure (`C=1CCCCC-1`), a bond or `.` with no atom after it (`CC=`), charges outside −15 to +15, and spaces anywhere in the string.

3.3 `style`

Journal style presets approximate the corresponding ChemDraw document stylesheets using the journals' published drawing settings — bond length, atom-label size, line width, a Helvetica/Arial font stack, and monochrome atom labels by default:

Preset	Bond	Line	Labels
`"acs"`	14.4 pt	0.6 pt	10 pt
`"rsc"`	12.2 pt	0.5 pt	7 pt
`"nature"`	10.8 pt	0.6 pt	6 pt
`"wiley"`	17 pt	0.7 pt	12 pt

```typst
#smiles("CC(N)C(=O)O") \
#smiles("CC(N)C(=O)O", style: "acs") \
#smiles("CC(N)C(=O)O", style: "nature") \
#smiles("CC(N)C(=O)O", style: "acs",
color: true)
```



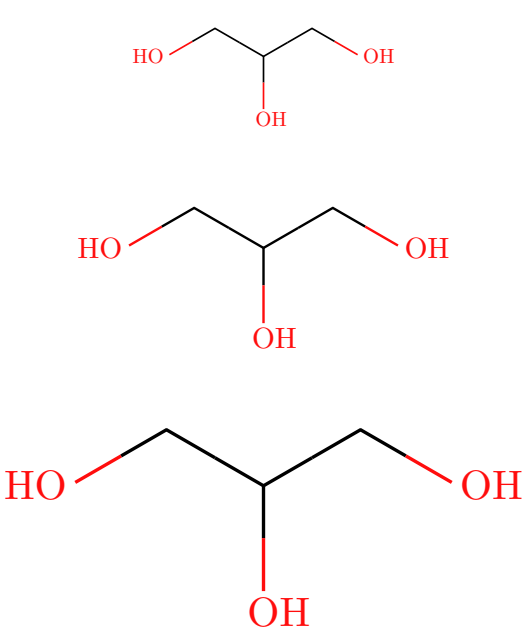
A preset only fills in arguments left unset: `style: "acs"` with `font-size: 14pt` keeps the ACS bond length under your label size, and `scale` multiplies the whole preset. `color: auto` makes journal presets monochrome; pass `color: true` for CPK colors or `color: false` for explicit black line art. `"default"` applies nothing, so existing documents are unaffected. Wiley's line width and label size are derived from its 6 mm bond-length guideline, which fixes only a minimum line width; journal-specific geometry beyond these four settings (double-bond spacing, hash spacing) keeps typed-smiles' defaults.

3.4 scale

Scales bond length, labels, and stroke proportionally.

```
#smiles("OCC(O)CO", scale: 0.7) \
#smiles("OCC(O)CO", scale: 1.0) \
#smiles("OCC(O)CO", scale: 1.4)
```

Typst

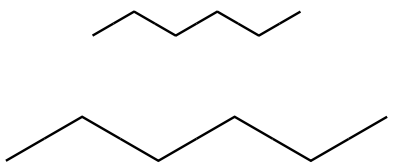


3.5 bond-length

Sets bond length on its own (`1.0` is 30 pt per bond), leaving label size and stroke under `scale`.

```
#smiles("CCCCCC", bond-length: 0.6) \
#smiles("CCCCCC", bond-length: 1.1)
```

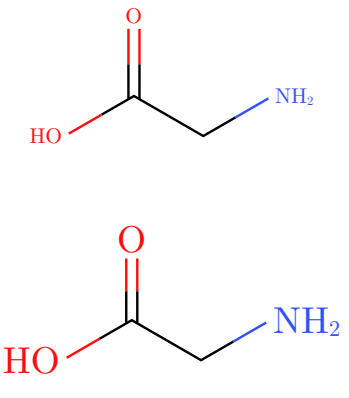
Typst



3.6 font-size

```
#smiles("OC(=O)CN", font-size: 8pt) \
#smiles("OC(=O)CN", font-size: 14pt)
```

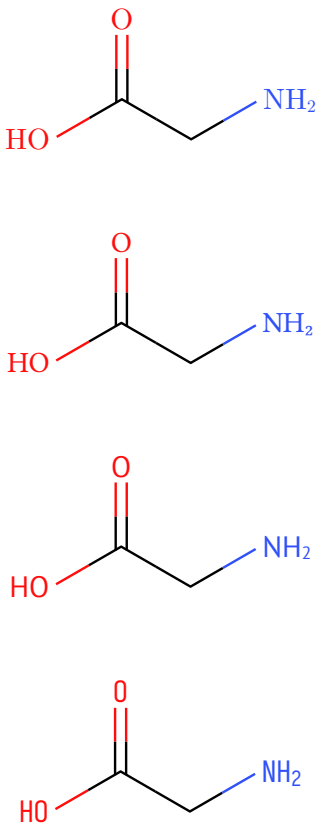
Typst



3.7 font

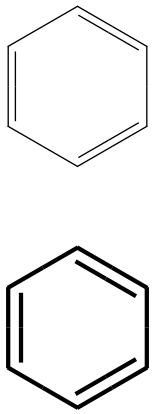
Match the document typeface, or pick something with character.

```typst
#smiles("OC(=O)CN", font: "New Computer Modern") \
#smiles("OC(=O)CN", font: "Libertinus Serif") \
#smiles("OC(=O)CN", font: "PT Sans") \
#smiles("OC(=O)CN", font: "Iosevka")
```



3.8 bond-stroke

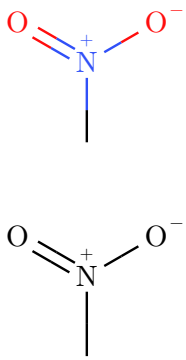
```typst
#smiles("C1=CC=CC=C1", bond-stroke: 0.5pt) \
#smiles("C1=CC=CC=C1", bond-stroke: 1.6pt)
```



3.9 color

CPK colors are on by default for `style: "default"` (see Section 8). Journal presets default to monochrome line art. Set `color: false` for an explicit monochrome diagram, or `color: true` to force CPK colors.

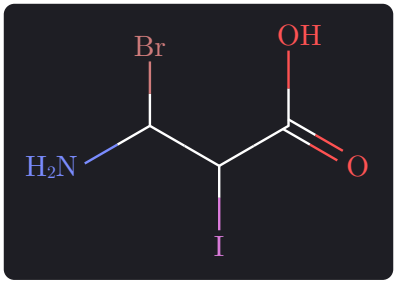
```typst
#smiles("C[N+](=O)[O-]") \
#smiles("C[N+](=O)[O-]", color: false)
```



3.10 fg and theme

Bond strokes and carbon labels follow `fg`, which defaults to `auto` and inherits the surrounding text color — on a dark slide theme (Touying, Polylux, or any `set text(fill: white)`) molecules recolor with no configuration. `theme: auto` switches to a dark CPK variant whenever the foreground is light, lifting only the hues that vanish on dark backgrounds (N, O, Br, I, and dark named label colors such as navy or maroon).

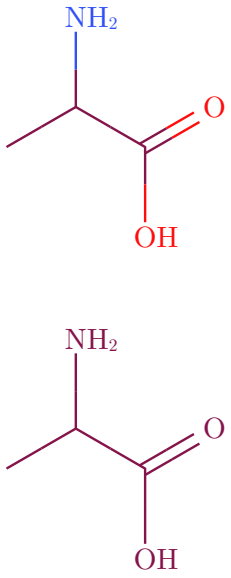
```typst
#block(fill: rgb("#1E1E24"),
inset: 8pt,
  radius: 4pt)[
  #set text(fill: white)
  #smiles("NC(Br)C(I)C(=O)O")
]
```



Both can be forced: an explicit `fg` recolors the skeleton on any background, and `theme: "dark"` or `theme: "light"` pins the palette. To set a document-wide default, rebind once in the preamble: `#let smiles = smiles.with(fg: white, theme: "dark")`.

```
#smiles("CC(N)C(=O)O", fg: maroon) \
#smiles("CC(N)C(=O)O", color: false, fg: maroon)
```

Typst



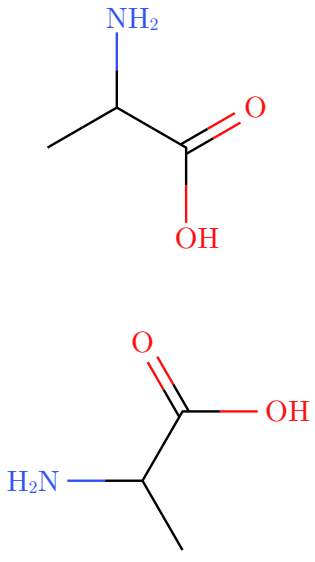
> **Note.** Reaction arrows inherit the surrounding text color the same way via `rxn-arrow(color: auto)`, so full schemes work on dark slides too.

3.11 `rotation`

Rotates the whole molecule while keeping every label upright.

```
#smiles("CC(N)C(=O)O", rotation: 0deg) \
#smiles("CC(N)C(=O)O", rotation: 90deg)
```

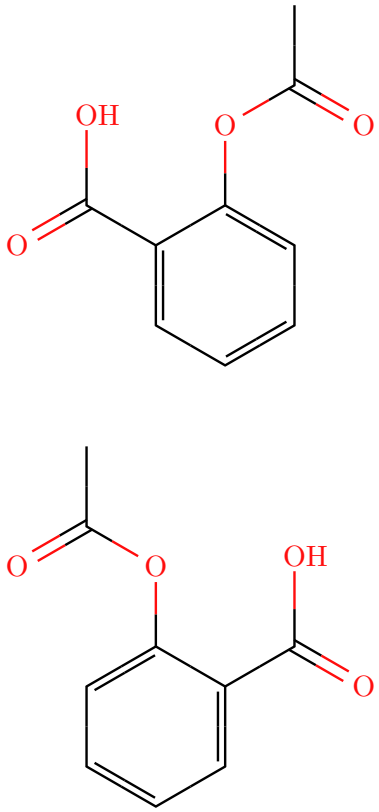
Typst



3.12 `mirror`

`mirror` reflects a molecule horizontally or vertically in the final drawing. Useful to face a reacting group toward a reaction arrow. Inside `reaction()`, pass it per molecule: `mol("OC1CCCCC1", mirror: "horizontal")`. With `rotation` set, `mirror: "vertical"` preserves left/right and flips top/bottom.

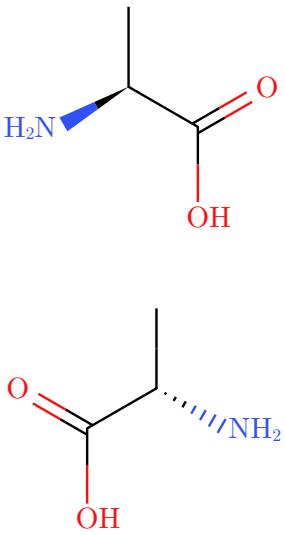
```typst
#smiles("CC(=O)OC1=CC=CC=C1C(=O)O") \
#smiles(
  "CC(=O)OC1=CC=CC=C1C(=O)O",
  mirror: "horizontal",
)
```



> **Note.** Mirroring preserves the depicted configuration at every stereocenter. Wedges and hashes change orientation as needed.

A stereocenter keeps its configuration when mirrored.

```typst
#smiles("N[C@@H](C)C(=O)O") \
#smiles(
  "N[C@@H](C)C(=O)O",
  mirror: "horizontal",
)
```

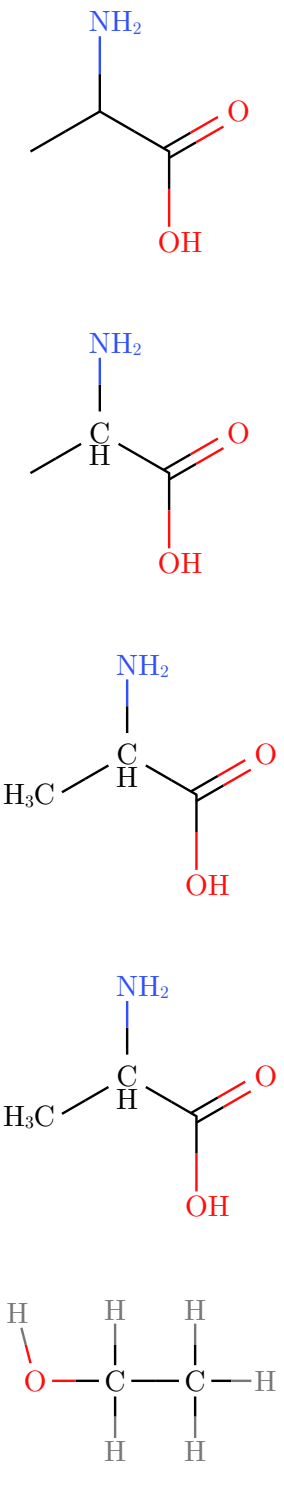


3.13 `show-h`

Hydrogens on heteroatoms are shown by default; implicit carbon hydrogens are hidden. `show-h` selects carbon hydrogens without needing a trailing comma for one atom: use an integer for one atom, an array for several atoms, or `"all"` for every implicit hydrogen. Use `"skeleton"` to draw every hydrogen as a separate atom with its own single bond. Eligible unbranched heavy-atom chains become straight rows; carbon-bound hydrogens use clean orthogonal directions, while lone pairs keep water bent and three-bond centers evenly spread in the displayed 2D schematic. Rings, branches, unsaturated paths, and stereochemical paths retain the normal heavy-atom layout. Ordinary molecule placement is unchanged. With the default color mode, element labels and bonds use the CPK palette; explicit skeleton and stereochemical H labels are gray unless `atom-colors: (H: color)` overrides them. Use `show-indices: true` to read off the indices while writing.

```typst
#smiles("CC(N)C(=O)O") \
#smiles("CC(N)C(=O)O", show-h: 1) \
#smiles("CC(N)C(=O)O", show-h: (0, 1)) \
#smiles("CC(N)C(=O)O", show-h: "all") \
#smiles("CCO", show-h: "skeleton")
```

Typst



4 Molecular formula: `mol-formula()`

`mol-formula(smiles)` parses the structure, counts explicit and implicit hydrogens, combines dot-separated components, and renders the result in Hill order. It returns content through the same chemformula renderer used by `ce()`.

```
Ethanol: #mol-formula("CCO") \
Benzene: #mol-formula("c1ccccc1") \
Sodium acetate: #mol-formula("CC(=O)[O-].[Na+]") \
Caffeine: #mol-formula("CN1C=NC2=C1C(=O)N(C(=O)N2C)C")
```

Typst

Ethanol: C_2H_6O
Benzene: C_6H_6
Sodium acetate: $C_2H_3NaO_2$
Caffeine: $C_8H_{10}N_4O_2$

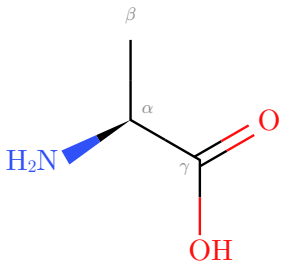
Dot-separated fragments are combined into one molecular composition and their formal charges are summed. This gives, for example, `C2H3NaO2` for sodium acetate and `ClH4N` for `[NH4+].[Cl-]`. A molecular formula is not an empirical formula, so subscripts are not reduced. Wildcards, custom labels, and isotope-labeled atoms fail with a descriptive error because their composition cannot be represented by this helper's ordinary-element output.

4.1 `atom-annotations`

Small annotations for NMR numbering, Greek positions, or footnote marks are drawn small and gray on the emptiest side of an atom. `atom-annotations` is always a tuple list. Each entry is either `(index, content)` or `(index, content, offset)`. Values are content, so wrap them in `text()` to restyle.

```
#smiles(
  "N[C@@H](C)C(=O)O",
  atom-annotations: (
    (1, [$alpha$], (0, -0.05)),
    (2, [$beta$]),
    (3, [$gamma$], (-0.4, -0.3)),
  ),
)
```

Typst

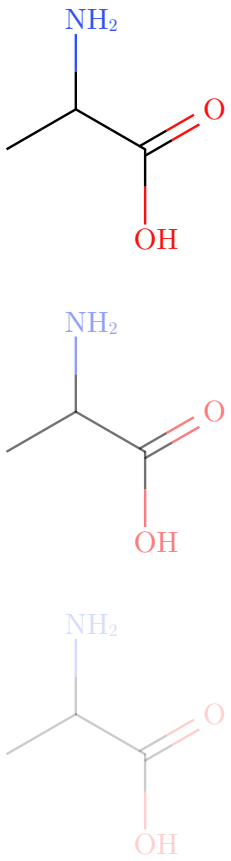


4.2 opacity

`opacity` fades every paint the drawing produces — bonds, atom labels, charges, lone pairs, and annotations — so a molecule can be ghosted or de-emphasized, e.g. spectator species in a mechanism or step-by-step slide builds. It accepts a ratio (`40%`) or a number between 0 and 1.

```
#smiles("CC(N)C(=O)O")
#h(1.5em)
#smiles("CC(N)C(=O)O", opacity: 55%)
#h(1.5em)
#smiles("CC(N)C(=O)O", opacity: 0.2)
```

Typst



Note. String molecules inside `reaction()` accept the same option: `mol("CCO", opacity: 30%)`.

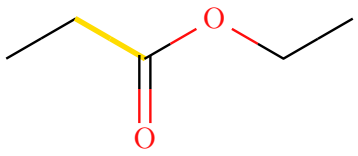
4.3 bond-customizations

`bond-customizations` restyles individual bonds. Each entry pairs a `bond(i, j)` reference (the same atom-index form used by `arrow()` and `highlight()`; a plain `(i, j)` array also works) with an options dictionary:

- `color` — draw the whole bond in one color instead of the bicolor halves,
- `stroke` — the bond width, like a per-bond `bond-stroke`,
- `opacity` — fade just this bond.

Overrides apply to every part of the bond: both lines of a double bond, the hash lines of a stereo bond, waves and dashes.

```typst
#smiles(
  "CCC(=O)OCC",
  bond-customizations: (
    (bond(1, 2), (color: yellow, stroke: 2pt)),
  ),
)
```

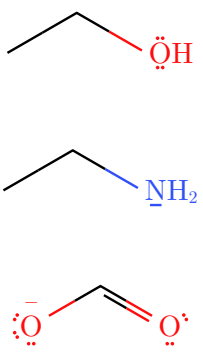


Note. Use `show-indices: true` while writing the references, exactly as for mechanism arrows. `bond-customizations` does not change bond length or angles; use `bond-length` to size all bonds together.

4.4 lone-pairs

Set `lone-pairs` to `"dots"` or `"lines"` to annotate the skeletal drawing with non-bonding electron pairs on common organic heteroatoms and charged atoms. The default `none` keeps lone pairs hidden.

```typst
#smiles("CCO", lone-pairs: "dots") \
#smiles("CCN", lone-pairs: "lines") \
#smiles("[O-]C=O", lone-pairs: "dots")
```



Note. Lone pairs are inferred for a conservative organic subset such as N, O, S, P, halogens, and simple charged forms. They are an annotation on the skeletal structure; the carbon skeleton and implicit-hydrogen conventions do not change. For example, ammonium (`[NH4+]`) has no nitrogen lone pair. On terminal hydrogen-bearing labels such as `OH`, `OH-`, and `NH2`, lone pairs are placed from the rendered heavy-atom glyph center. For charged bare atoms such as `[O-]` or `[Br-]`, the charge mark does not move the atom center used for lone pairs. Lone pairs are never inferred for `{label}` abbreviations, whose internal bonds are hidden; declare them explicitly with `lp=N` — see Section 9.4.

5 Atoms and charges

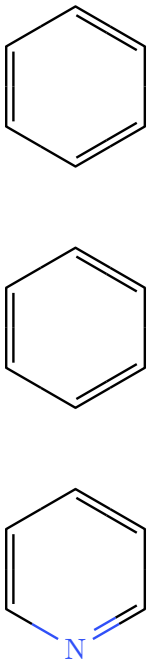
Standard SMILES syntax (atoms, bonds, branches, ring closures) is supported. This section covers the points specific to typed-smiles.

> **Note.** Ring closures may be written next to branch points. For example, `C1=CCCC(=O)1` closes the ring on the same atom that carries the C = O branch.

5.1 Aromatic rings

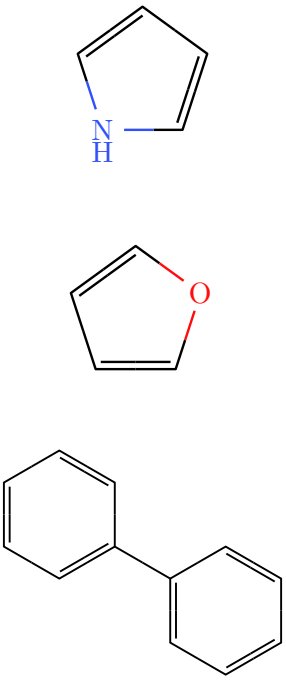
Aromatic rings can be written in either OpenSMILES form: lowercase aromatic atoms (`c1ccccc1`) or Kekulé notation with explicit alternating double bonds (`C1=CC=CC=C1`). Both forms produce the same skeletal depiction.

```typst
#smiles("c1ccccc1") \
#smiles("C1=CC=CC=C1") \
#smiles("c1ccncc1")
```



Bracket aromatic atoms carry their hydrogen count explicitly, as in pyrrole's `[nH]`. A single bond between two aromatic rings (biphenyl) may be written with or without the explicit `-`.

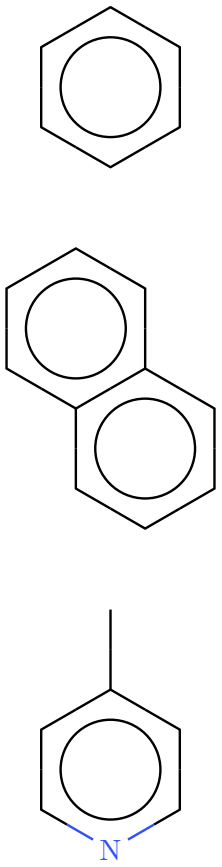
```typst
#smiles("c1cc[nH]c1") \
#smiles("c1occc1") \
#smiles("c1ccccc1-c1ccccc1", scale: 0.8)
```



`aromatic: "circle"` depicts lowercase-aromatic rings as single bonds with an inscribed circle instead of alternating double bonds. Each fully aromatic ring of a fused system gets its own circle; Kekulé-written input always keeps its explicit bonds.

```
#smiles("c1ccccc1", aromatic: "circle") \
#smiles("c1ccc2ccccc2c1", aromatic: "circle") \
#smiles("Cc1ccncc1", aromatic: "circle")
```

Typst



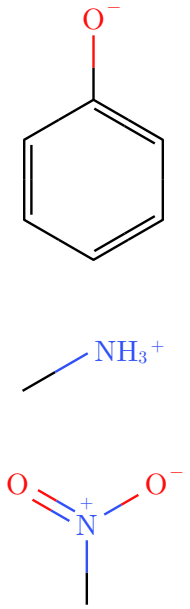
Note. Atom indices follow SMILES writing order for both aromatic and Kekulé input, so the same `show-indices`, `highlight(...)`, and `arrow(...)` references apply to either form. Write aromatic hydrogens explicitly, as in `[nH]`; invalid aromatic rings or aromatic atoms outside a ring produce an error.

5.2 Charges

Formal charges inside brackets render as a raised sign after the atom. Hydrogen counts and charge marks use the atom-label size so they remain legible when labels are scaled.

```
#smiles("[O-]C1=CC=CC=C1") \
#smiles("C[NH3+]") \
#smiles("C[N+](=O)[O-]")
```

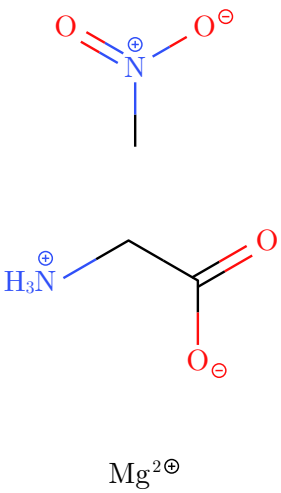
Typst



`charge-style: "circled"` encloses each sign in a circle, as in organic structure drawings. A charge larger than one writes its size before the circle. The option also works on `mol()`, `smiles-inline()`, and `smiles-cetz()`.

```
#smiles("C[N+](=O)[O-]", charge-style: "circled") \
#smiles("[NH3+]CC(=O)[O-]", charge-style: "circled") \
#smiles("[Mg+2]", charge-style: "circled")
```

Typst



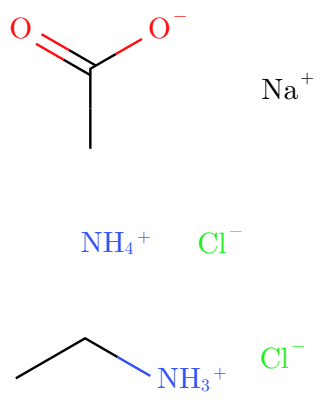
Note. A bracketed element such as `[N]` is a nitrogen atom. To print an arbitrary text label instead, use the abbreviation syntax `{N}` (see Section 9).

5.3 Salts and disconnected structures

A dot (`.`) means “no bond”: it separates disconnected fragments, as in salts, counterions, and hydrates. Each fragment is laid out on its own and the fragments are arranged left to right in writing order.

```
#smiles("CC(=O)[O-].[Na+]") \
#smiles("[NH4+].[Cl-]") \
#smiles("CC[NH3+].[Cl-]")
```

Typst



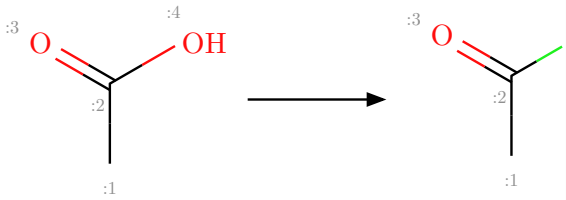
Note. Atom indices stay global across fragments, still following SMILES writing order, so `show-indices`, `highlight(...)`, and `arrow(...)` can reference atoms in any fragment of the same `smiles()` call. A ring closure written across a dot (as in the OpenSMILES example `C1.C1`, ethane) still forms its bond.

5.4 Atom maps

A number after a colon at the end of a bracket atom is an OpenSMILES atom class, commonly used as an atom map in reaction schemes: `[CH3:7]`. Maps never change the structure or the atom indices used by `atom()` references. Pass `show-maps: true` to label each mapped atom; map 0 means “unmapped”, as in OpenSMILES.

```
#reaction(
  mol("[CH3:1][C:2](=[O:3])[OH:4]", show-maps: true),
  rxn-arrow(),
  mol("[CH3:1][C:2](=[O:3])Cl", show-maps: true),
)
```

Typst



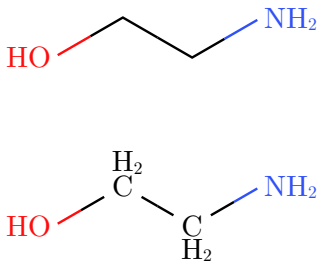
Note. An `atom-annotations` entry for a mapped atom replaces its map label, so you can restyle individual maps.

6 Hydrogens

6.1 Default display

Heteroatom hydrogens are shown by default; implicit carbon hydrogens are hidden for a clean skeleton. `show-h: "all"` labels carbon hydrogens as well.

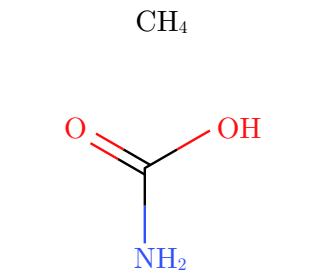
```typst
#smiles("OCCN") \
#smiles("OCCN", show-h: "all")
```



6.2 Explicit bracket hydrogens

A hydrogen written as its own bracket atom (for example the `[H]` atoms in `C([H])([H])[H]`) is folded into its neighbor's hydrogen count, exactly like an implicit hydrogen. Carbon-bound hydrogens stay hidden and heteroatom hydrogens are still labeled, so a fully hydrogen-suppressed SMILES depicts the same as its skeletal form. Hydrogen counts written inside a single bracket are shown on heteroatoms, such as `[OH-]` or `[NH4+]`, and on labeled isotope or charged carbons. Neutral carbon hydrogens follow the skeletal convention; request them with `show-h` when needed.

```typst
#smiles("C([H])([H])([H])[H]") \
#smiles("N([H])([H])C(=O)O")
```

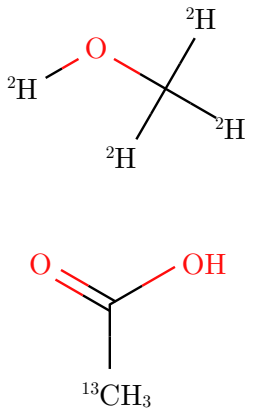


6.3 Isotopes

A bracket isotope is drawn as a leading superscript mass number. Isotopically labeled hydrogens such as deuterium `[2H]` are kept as drawn atoms rather than folded away.

```
#smiles("[2H]OC([2H])([2H])[2H]")
\
#smiles("[13CH3]C(=O)O")
```

Typst

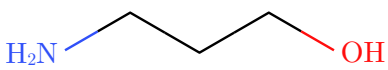


6.4 Label orientation

Terminal heteroatom labels orient so the bond meets the heavy atom rather than the trailing hydrogen, at any angle.

```
#smiles("NCCCO")
```

Typst



7 Stereochemistry

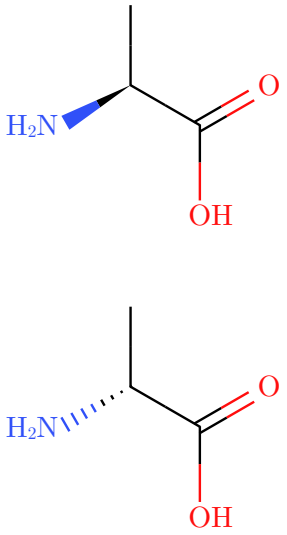
typed-smiles supports both standard SMILES stereo notations, plus drawing extensions for bond styles and local acyclic-chain layout.

7.1 Tetrahedral centers: `@` and `@@`

A bracket atom carrying `@` or `@@` becomes a wedge (toward the viewer) or hashed bond (away) in the conventional SMILES orientation. The narrow end of every wedge sits on the stereocenter it describes.

```
1  #smiles("N[C@@H](C)C(=O)O") \
2  #smiles("N[C@H](C)C(=O)O")
```

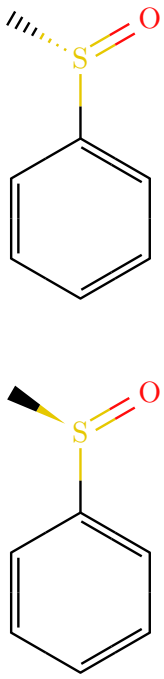
Typst



A center with three neighbors and a lone pair, such as the sulfur of a sulfoxide or a phosphine phosphorus, is drawn the same way; the lone pair takes the place of the fourth neighbor.

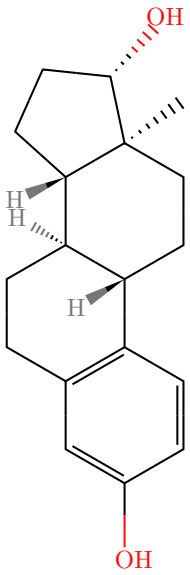
```
1  #smiles("C[S@](=O)c1ccccc1") \
2  #smiles("C[S@@](=O)c1ccccc1")
```

Typst



Write the stereocenter with `@` or `@@`. An exocyclic substituent such as OH or CH_3 carries the wedge when present; otherwise the explicit hydrogen does.

```
#smiles(
  "C[C@]12CC[C@H]3[C@H]
  ([C@@H]1CC[C@@H]2O)CCC4=C3C=CC(=C4)O",
  scale: 0.85,
)
```

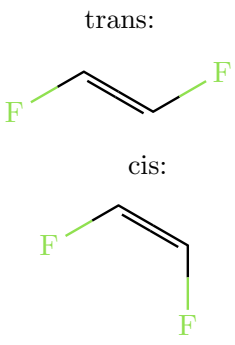


Note. R/S descriptors are not available. Ring stereochemistry between adjacent centers may need a manual adjustment (see Section 19).

7.2 Double bonds: / and \

Directional bonds `/` and `\` around a double bond set its cis/trans geometry. They come in pairs, one on each side of the `=`. If an alkene carbon has two marked substituent bonds, the markers must place those substituents on opposite sides.

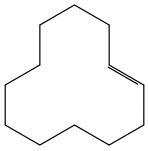
```
trans: #smiles("F/C=C/F")
#h(2em)
cis: #smiles("F/C=C\F")
```



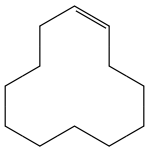
Markers on a ring double bond set the geometry of the ring itself. A trans double bond needs a ring of at least nine atoms; a smaller ring cannot show it in a flat drawing, so it is reported like other stereochemistry the drawing cannot show (see Section 7.4).

```typst
trans: #smiles("C1CCCCC/C=C/CCCC1", scale: 0.5)
#h(2em)
cis: #smiles("C1CCCCC/C=C\CCCC1", scale: 0.5)
```

trans:



cis:

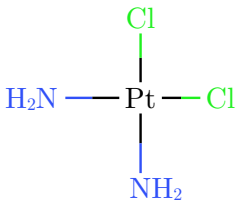


7.3 Square-planar centers: `@SP1`, `@SP2`, `@SP3`

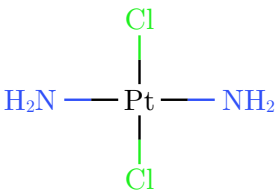
Square-planar stereochemistry is depicted exactly: the four neighbors sit at 90° around the center, arranged so the line traced through them in SMILES order forms the class's shape — 'U' for `@SP1`, '4' for `@SP2`, 'Z' for `@SP3`. The classic example is cis- versus trans-platin.

```typst
cis: #smiles("N[Pt@SP1](N)(Cl)Cl")
#h(2em)
trans: #smiles("N[Pt@SP2](N)(Cl)Cl")
```

cis:



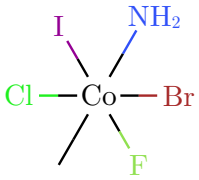
trans:



7.4 Stereochemistry the drawing cannot show

Some written stereochemistry has no faithful 2D depiction here: trigonal-bipyramidal (`@TB1`–`@TB20`), octahedral (`@OH1`–`@OH30`), and allene (`@AL1`, `@AL2`) centers, a `@` on an atom that cannot be a stereocenter (such as `[C@H2]`), and configurations the layout cannot place, such as a trans double bond in an eight-membered ring. By default these are errors, so a figure never drops stereochemistry silently. Pass `undepicted-stereo: "omit"` to draw the structure without it.

```
#smiles(
  "C[Co@OH1](F)(Cl)(Br)(I)N",
  undepicted-stereo: "omit",
)
```

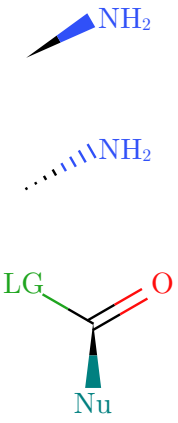


> **Note.** To accept undrawn stereochemistry throughout a document, set the option once with `smiles.with(undepicted-stereo: "omit")` (see the section on project-wide defaults). It applies to `smiles()`, `mol()`, and `smiles-cetz()`.

7.5 Manual wedges: `!w` and `!h`

For direct control, `!w` draws a solid wedge and `!h` a hashed wedge. These are typed-smiles extensions, not standard SMILES. Like plain bonds, they are colored by the atoms at each end.

```
#smiles("C!wN") \
#smiles("C!hN") \
#smiles("C(!w{Nu|teal})({LG|green})=O")
```

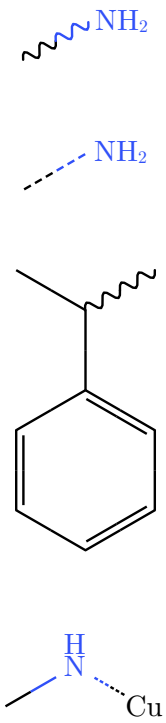


7.6 Wavy and dashed bonds: `!s` and `!d`

Two more drawing extensions on single bonds: `!s` draws a wavy (squiggly) bond — the conventional depiction of unspecified stereochemistry or an attachment point — and `!d` draws a dashed bond for hydrogen bonds, partial bonds, and coordination. Like plain bonds, they are colored by the atoms at each end.

```
#smiles("C!sN") \
#smiles("C!dN") \
#smiles("CC(!sC)C1=CC=CC=C1") \
#smiles("CN!d[Cu]")
```

Typst



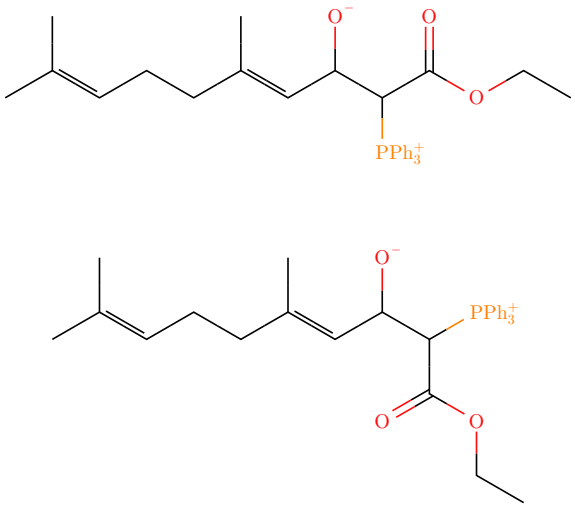
Note. `!s` and `!d` carry no cis/trans meaning and never consume a double-bond geometry slot; they can appear next to real `/` and `\` directional bonds.

Note. Tip: Use empty custom labels `{}` or `!w{}` to draw bonds without a connecting atom.

7.7 Curling an acyclic chain: `!c`

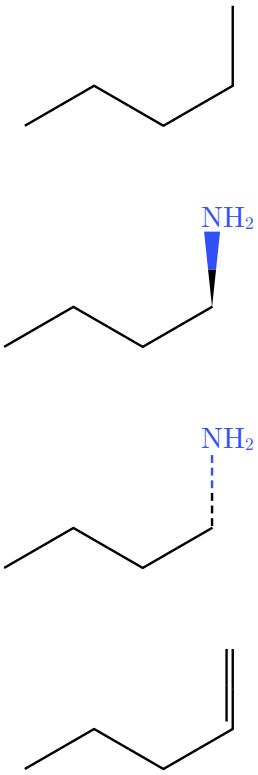
A normal skeletal chain alternates its left and right turns. `!c` repeats the preceding turn for the next bond, curling the forward chain and any sibling branches as a unit while preserving ideal 120° bond angles. It is a drawing extension only: the molecule's connectivity and stereochemistry do not change.

```typst
#smiles("CC(C)=CCCC(C)=CC([O-])C({>PPh_3^+ | P})C(=O)OCC", scale: 0.68) \
#smiles("CC(C)=CCCC(C)=CC([O-])C({>PPh_3^+ | P})!cC(=O)OCC", scale: 0.68)
```



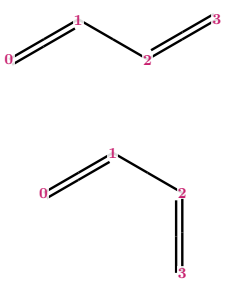
`!c` is a layout modifier and composes with drawing styles or an explicit bond order. The canonical order is curl, style, then bond order.

```typst
#smiles("CCCC!cC") \
#smiles("CCCC!c!wN") \
#smiles("CCCC!c!dN") \
#smiles("CCCC!c=C")
```



A conjugated diene is drawn s-trans by default, with its termini on opposite sides of the central bond. Put `!c` on the second double bond to draw the s-cis conformer: the first bond supplies the turn that the curl repeats, so both termini sit on the same side.

```typst
#smiles("C=CC=C", show-indices: true) \
#smiles("C=CC!c=C", show-indices: true)
```



Note. `!c` needs two chain bonds in front of it, so there is an established turn to repeat: the first and second bonds of a chain cannot be curled. For an s-cis 1,3-diene, write the curl on the second double bond as `C=CC!c=C` the spelling `C=C!cC=C` fails. It is not supported on a ring bond or directly after one. Only one `!c` constraint may leave a given atom.

8 Colors

8.1 Jmol CPK palette

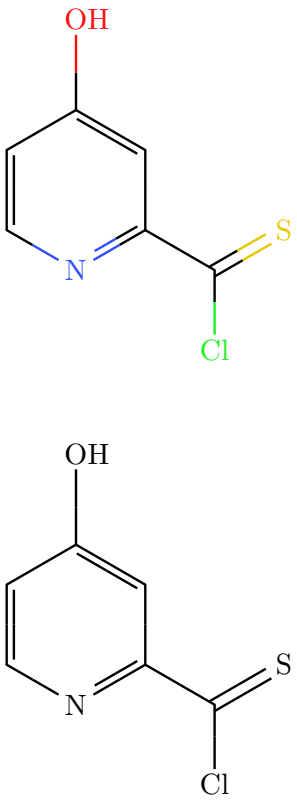
Atoms use the Jmol CPK palette by default. Carbon and unlisted elements are black; common heteroatoms get their conventional colors.

N	O	S	P	F	Cl	Br	I

Set `color: false` for a monochrome diagram.

```
#smiles("OC1=CC(=NC=C1)C(=S)Cl")
\
#smiles("OC1=CC(=NC=C1)C(=S)Cl", color: false)
```

Typst



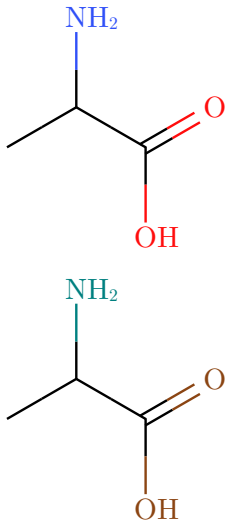
8.2 Custom colors (`atom-colors`)

`atom-colors` is a single dictionary that overrides colors for both elements and arbitrary abbreviated groups. Any Typst color value works — `rgb()`, `luma()`, named constants, or anything else.

Element-symbol keys

Use an element symbol as the key to replace that element's CPK color everywhere it appears, including when referenced as an inline label style (`{OMe|O}` would pick up the overridden oxygen color).

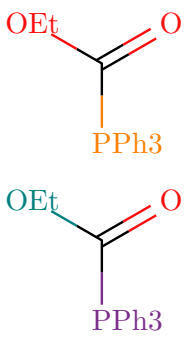
```typst
// default CPK
#smiles("CC(N)C(=O)O")
// oxygen → dark brown; nitrogen → teal
#smiles("CC(N)C(=O)O",
  atom-colors: (O: rgb("#8B4513"), N: rgb("#008080")))
```



Label-name keys

Wrap the label text in braces to target a specific abbreviated group, regardless of any inline style it carries. The key is written as a quoted string because `{` is not a valid Typst identifier character.

```typst
// default — colors come from inline styles or element fallback
#smiles("{>PPh3|P}C({OEt|O})=O")
// override by label name — inline style is ignored for these
#smiles("{>PPh3|P}C({OEt|O})=O",
  atom-colors: ("{PPh3}": rgb("#7B2D8B"), "{OEt}": rgb("#008080")))
```

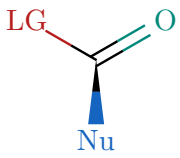


Mixing both key forms

Element keys and label keys can coexist in the same dictionary.

```
// O (element) → teal; {Nu} (label) → navy; {LG} (label) → maroon
#smiles("C(!w{Nu})({LG|red})=O",
  atom-colors: (O: rgb("#00897B"), "{Nu}": rgb("#1565C0"), "{LG}": rgb("#B71C1C")))
```

Typst



Priority

Color is resolved in this order for each atom or label:

1. Label-name key in `atom-colors` (e.g. `"{PPh3}"`), if the atom is an abbreviation.
2. Element-symbol key in `atom-colors` (e.g. `O`), for both real atoms and element-style labels.
3. The inline `{label|style}` style from the SMILES string (named color, hex, or element symbol).
4. The default CPK palette.

> **Note.** `color: false` is a hard override that sits above all of the above. When it is set, every atom and label uses the foreground `fg` regardless of any `atom-colors` entries or inline styles. If you want a mostly-monochrome diagram with one or two highlighted groups, keep or set `color: true` and put everything else in `atom-colors`.

8.3 Label colors (`{label|style}`)

In the SMILES string, write `{label|style}` to color an abbreviated group independently of its element. The style can be a named color, an element symbol, or a hex code.

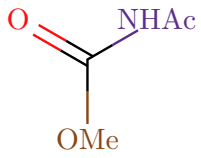
Named colors:

Name	Swatch	Name	Swatch
red		orange	
yellow		brown	
green		lime	
teal		cyan	
blue		navy	
purple		pink	
black		gray	
silver		maroon	
white			

Hex colors: write a `#RRGGBB` hex code after the pipe to use any color. The `#` is just a regular character inside a Typst string literal.

```
1  #smiles("{OMe|#8B4513}C(=O){NHAc|
   #5B2E8C}")
```

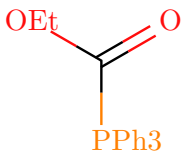
Typst



Element symbol: using a symbol as the style applies that element's (possibly overridden) CPK color to the label and its bonds.

```
1  #smiles("{>PPh3|P}C({OEt|O})=O")
```

Typst



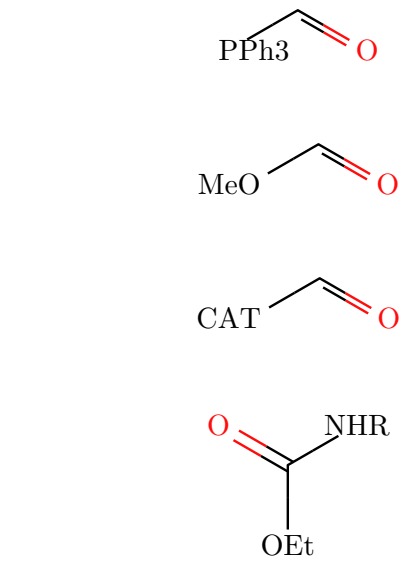
9 Abbreviated groups

9.1 Plain labels

Wrap text in braces `{...}` to place a labeled pseudo-atom that bonds like any other atom. Use `>` inside the label to choose the attachment glyph; the marker is not displayed. Rotate the molecule when needed so the bond approaches the chosen glyph roughly perpendicular to the written label.

```
#smiles("{>PPh3}C=O") \
#smiles("{Me>O}C=O") \
#smiles("{CA>T}C=O") \
#smiles("{OEt}C(=O){NHR}")
```

Typst

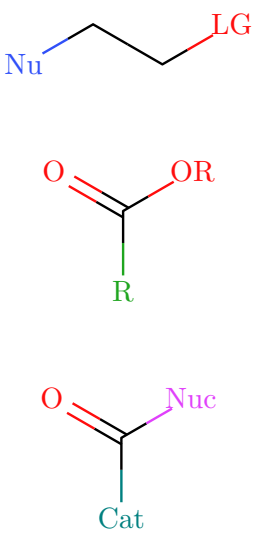


9.2 Colored labels

Add `|style` inside the braces to color a label. Use a named color, an element symbol, or a hex code — see Section 8 for the full list.

```
#smiles("{Nu|blue}CC{LG|red}") \
#smiles("{R|green}C(=O){OR|O}") \
#smiles("{Cat|teal}C(=O){Nuc|#E040FB}")
```

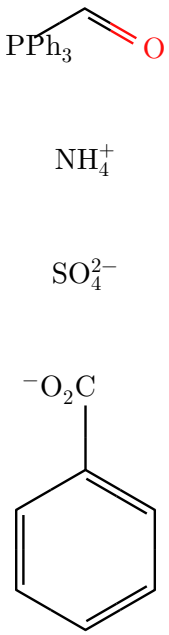
Typst



9.3 Scripted labels

Use `_(...)`, `^(...)`, or their one-character forms `_4` and `^+` for explicit subscripts and superscripts in a label. When both follow one glyph, they are paired with that glyph: `{NH_4^+}` places the plus above H rather than above 4. Script size, spacing, and vertical placement match `ce()` notation. Parenthesize multi-character scripts. A script at the very start of a label has no glyph before it, so it is drawn ahead of the first glyph: `{^-O_2>C}` reads $^{-}O_2C$.

```typst
#smiles("{>PPh_(3)}C=O") \
#smiles("{NH_4^+}") \
#smiles("{SO_(4)^(2-)}") \
#smiles("{^-O_2>C}c1ccccc1")
```



9.4 Lone pairs on labels

A label hides its internal bonds, so lone pairs cannot be inferred from its text. Declare them explicitly with an inline `lp=N` modifier, where `N` is an integer from 1 to 4. The grammar of a label body is:

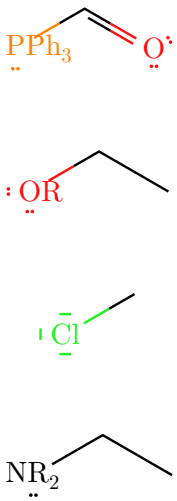
```
{text[|style][|modifier]...}
```

The optional style must come before named modifiers. Each modifier gets its own pipe-delimited field. As with all lone pairs, the `lone-pairs` argument controls whether they are drawn and whether they appear as dots or lines — `lp=N` only supplies the count.

When the label has a `>` attachment marker the pairs belong to the marked glyph; otherwise they surround the whole label. In both cases the pairs are kept clear of the rest of the label and of the bonds, and their placement is deterministic under rotation.

```
#smiles("{>PPh_3|P|lp=1}C=O", lone-pairs: "dots") \
#smiles("{>OR|O|lp=2}CC", lone-pairs: "dots") \
#smiles("{>Cl|Cl|lp=3}C", lone-pairs: "lines") \
#smiles("{NR_2|lp=1}CC", lone-pairs: "dots")
```

Typst



Note. A custom-label lone pair is addressable with `lp(i)` exactly like an inferred one, so an electron-pushing `arrow()` in a mechanism can start from it. The drawn pair and the arrow tail come from the same geometry, so they coincide.

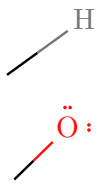
9.5 Manual label offsets

Add `offset=(x, y)` to move one custom-label atom after automatic molecular layout. Values are measured in bond-length units. Positive x moves right and positive y moves up on the page, even when `rotation` is set.

The label, every bond endpoint incident to that atom, its lone pairs, indices, annotations, highlights, and mechanism references move together. Other atom coordinates and the automatic layout remain unchanged, so a large manual displacement can intentionally create overlaps.

```
#smiles("C{H|grey|offset=(0.1, 0.2)}")
#smiles(
  "C{>O|O|lp=2|offset=(-0.2, 0.15)}",
  lone-pairs: "dots",
)
```

Typst



Combine style, lone pairs, and displacement by giving each one its own field:

```
{H|grey|lp=1|offset=(0.1, 0.1)}
```

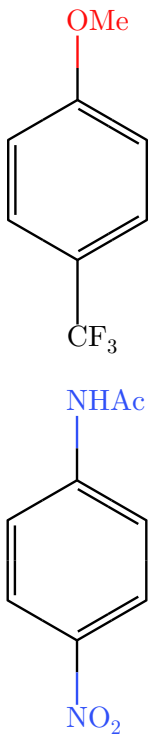
The named `lp=` and `offset=` fields may be exchanged; the optional plain style field must remain before both.

9.6 Automatic abbreviations

Write the full structure and let `abbreviate` draw common terminal groups as labels. Pass catalogue names, or `"all"` for every catalogue group that occurs. Groups are recognized from the molecular structure, not the SMILES text, so any atom order that encodes the same molecule gives the same drawing.

```
#smiles(
  "COc1ccc(C(F)(F)F)cc1",
  abbreviate: ("OMe", "CF3"),
)
#smiles(
  "CC(=O)Nc1ccc(cc1)[N+](=O)[O-]",
  abbreviate: "all",
)
```

Typst



Name	Matched terminal group	Labels

`"tBu"`	Quaternary carbon carrying three methyl groups	tBu
`"CO2Et"`	Ethyl ester: carbonyl carbon carrying an ethoxy group	CO_2Et / EtO_2C
`"CO2Me"`	Methyl ester: carbonyl carbon carrying a methoxy group	CO_2Me / MeO_2C
`"OAc"`	Acetoxy: oxygen carrying an acetyl group	OAc / AcO
`"NHAc"`	Acetamido: NH carrying an acetyl group	NHAc / AcHN
`"SO3H"`	Sulfonic acid, written `S(=O)(=O)O`	SO_3H / HO_3S
`"CF3"`	Carbon carrying three fluorines	CF_3 / F_3C
`"NO2"`	Nitro group, written `[N+](=O)[O-]` or `N(=O)=O`	NO_2 / O_2N
`"CO2H"`	Carboxylic acid: carbonyl carbon carrying an OH	CO_2H / HO_2C
`"CO2-"`	Carboxylate with its charge shown in the label, written `C(=O)[O-]`	CO_2^- / ^-O_2C
`"CN"`	Nitrile carbon triple-bonded to nitrogen	CN / NC
`"CHO"`	Aldehyde: carbonyl carbon carrying one hydrogen	CHO / OHC
`"OEt"`	Oxygen carrying an ethyl group	OEt / EtO
`"OMe"`	Oxygen carrying a methyl group	OMe / MeO
`"Ac"`	Carbonyl carbon carrying a methyl group	Ac

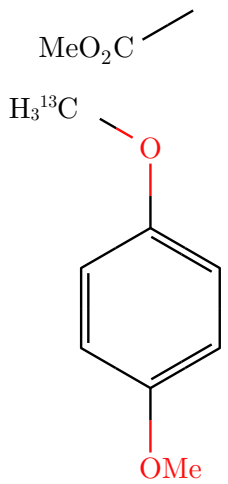
Each group attaches to the rest of the molecule through one single bond. The label reads away from the molecule; when its bond leaves toward the right after `rotation` and `mirror`, it reads toward the bond instead (MeO, F_3C).

What stays expanded. A label must not hide meaning it cannot show, so a group stays drawn in full when any of its atoms carries an isotope, an atom map, a stereo mark, an unexpected charge, a bracket hydrogen count, or a `{label}`, or when an internal bond carries a direction or drawing extension. The attachment bond itself may carry stereo marks; stereocenters next to a label keep their configuration.

Overlaps. When candidates share atoms, the catalogue order of the table decides, whatever order the request lists. Larger groups come first, so CO_2Me wins over OMe and Ac in a methyl ester, NHAc over Ac in an acetanilide, and OEt over OMe in an ethyl methyl ether. When the ester is not requested, OMe still wins over Ac. A group never attaches to another automatic label. A group named explicitly must be drawn at least once, or the call fails; `"all"` silently skips absent groups.

```
1  #smiles("COC(C)=O", abbreviate:
   "all")
2  #smiles("[13CH3]Oc1ccc(OC)cc1",
   abbreviate: "all")
```

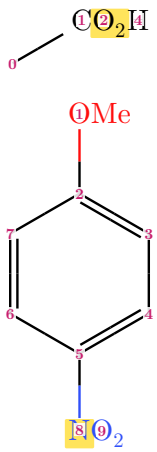
Typst



References. Formula abbreviations such as CO_2H, SO_3H, CF_3, NO_2, and CN expose one representative original index per displayed element fragment. An O_2 or F_3 fragment has one badge, and `highlight(atom(i))` covers the element and its subscript. The H fragment retains its original hydrogen index. Shorthand labels such as OMe remain single selectable groups at their attachment atom's index.

Warning: visible indices can have gaps. Abbreviation reduces the number of displayed fragments. Indices represented by another fragment are skipped so that the remaining atom mappings stay intact. For `CC(=O)O`, CO_2H displays C at index 1, O_2 at 2, and H at 4; index 3 has no separate badge. Use `show-indices: true` to inspect the selectable indices. A skipped atom cannot be targeted individually: remove the group from `abbreviate` to address it. Internal bonds and lone pairs remain unavailable, and `show-h` and `atom-annotations` entries for hidden atoms are rejected.

```typst
#smiles("CC(=O)O", abbreviate:
"CO2H", show-indices: true,
highlight(atom(2)))
#smiles(
  "COc1ccc(cc1)[N+](=O)[O-]",
  abbreviate: "all",
  show-indices: true,
  highlight(atom(8)),
)
```



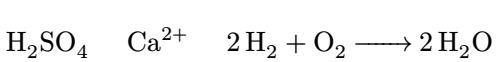
Note. Formulas and masses always use the full structure: `mol-formula` and `mol-weight` take the same SMILES string and ignore the drawing. `atom-colors` keys such as `"{OMe}"` color an automatic label in both reading directions. `mol()`, `smiles-inline()`, and `smiles-cetz()` accept `abbreviate` too; `smiles-cetz()` registers anchors for selectable formula fragments, but no `atom-<i>` anchor for skipped atoms.

10 Chemical formulas: ce()

For formulas and text equations, ce is re-exported from the chemformula package, so one import covers both structures and formulas.

```
#ce("H2SO4") #h(1em) #ce("Ca^2+")
#h(1em) #ce("2 H2 + O2 -> 2 H2O")
```

Typst

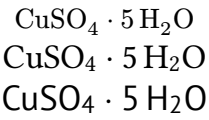


10.1 Fonts and size

ce accepts font, font-size, charge-style, and stacked-charges (see below); any other arguments pass through to chemformula.

```
#ce("CuSO4 * 5 H2O") \
#ce("CuSO4 * 5 H2O", font: "Libertinus
Serif", font-size: 13pt) \
#ce("CuSO4 * 5 H2O", font: "PT Sans",
font-size: 13pt)
```

Typst



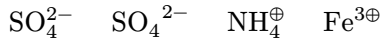
10.2 Charge placement and style

Argument	Type	Default	Description
stacked-charges	bool	true	Stack a charge over the subscript that follows the same atom, as in SO4^2-. With false the charge follows the subscript.
charge-style	"plain" / "circled"	"plain"	Write charges as a bare sign, or with the sign inside a circle and the size of a larger charge before it.

Both options change only written charges such as ^2-, +, and -; isotopes, exponents, and other scripts keep their place.

```
#ce("SO4^2-") #h(1em)
#ce("SO4^2-", stacked-charges: false)
#h(1em)
#ce("NH4+", charge-style: "circled")
#h(1em)
#ce("Fe^3+", charge-style: "circled")
```

Typst



11 Molecular weight: `mol-weight()`

`mol-weight(smiles)` returns the molecular weight in g/mol as a plain `float`: the sum of rounded atomic weights from the element table over every atom, including implicit and explicit hydrogens. Round it for display with `calc.round`.

```typst
Ethanol: #calc.round(mol-weight("CCO"), digits: 2) g/mol \
Caffeine: #calc.round(
  mol-weight("CN1C=NC2=C1C(=O)N(C(=O)N2C)C"),
  digits: 2,
) g/mol \
Sodium acetate: #calc.round(
  mol-weight("CC(=O)[O-].[Na+]"),
  digits: 2,
) g/mol
```

Typst

Ethanol: 46.07 g/mol
Caffeine: 194.19 g/mol
Sodium acetate: 82.03 g/mol

Dot-separated fragments are summed together, so salts and hydrates work as written. Charges are accepted; as in standard formula-weight arithmetic, the electron mass difference of ions is ignored.

> **Note.** Compilation fails with a descriptive error when the weight is undefined: wildcard `*` atoms, `{label}` abbreviations (no defined composition), and isotope-labeled atoms such as `[2H]` (a specific nuclide needs a nuclide mass, not a standard atomic weight).

12 Inline molecules: `smiles-inline()`

`smiles-inline(smiles-str, height: 1.4em, baseline: auto, ..args)` scales a structure to a target height and baseline-aligns it so it reads inline, like a word in the sentence. Extra named arguments are passed straight to `smiles()`.

```
Dehydrating ethanol #smiles-inline("CCO")
over alumina gives ethylene
#smiles-inline("C=C"); toluene
#smiles-inline("Cc1ccccc1") is a common
solvent.
```

Typst

Dehydrating ethanol  over alumina gives ethylene  toluene  is a common solvent.

Sizing works in two steps: the drawing is scaled to fit the `height` (default `1.4em`, so ordinary line spacing stays essentially unchanged), and the scale is capped so one bond never exceeds most of that height — otherwise a flat, mostly horizontal molecule, which measures almost no height, would blow up to fill it. Typst grows a line to fit tall inline content, so neighboring lines are never overlapped; passing a larger `height` only makes the molecule's own line taller.

> **Note.** `baseline` sets how far the drawing's vertical center sits above the text baseline (default centers it on the lowercase body). Atom labels scale down with the drawing: at inline sizes, label-heavy molecules read better with a larger `height` or as a display-mode `smiles()`.

13 CeTZ integration: `smiles-cetz()`

`smiles-cetz(smiles-str, name:, origin:, fg:, theme:, ..opts)` draws a molecule **inside an existing CeTZ canvas** and registers named anchors on the group, so arbitrary CeTZ drawing attaches to real molecular positions:

- `"<name>.atom-<i>"` — every atom center (writing-order indices, as shown by `show-indices`),
- `"<name>.bond-<i>-<j>"` — every bond midpoint (with $i < j$),
- `"<name>.center"` — the molecule origin.

Coordinates are in bond-length units; give the canvas `length: 30pt * scale` so sizes match `smiles(scale: ...)`, and wrap the canvas in `context` (atom-label measurement needs it — plain `smiles()` provides it automatically, a raw canvas does not).

The Watson–Crick A–T base pair below is two `smiles-cetz` calls plus plain CeTZ: two dashed hydrogen bonds drawn between atom anchors of the two bases, endpoint nudges so the dashed lines do not start at atom centers, distance labels, and glycosidic-bond stubs to the sugar backbone.

Typst

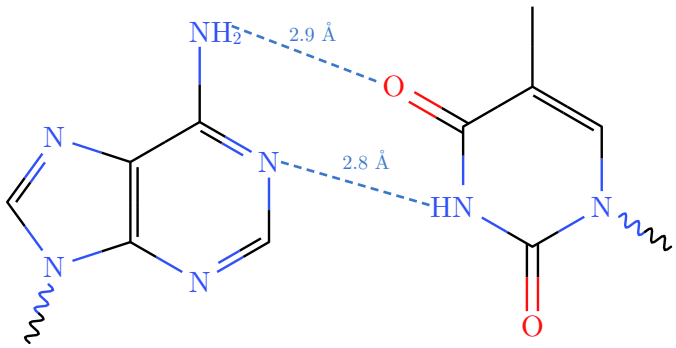
```
#align(center, context cetz.canvas(length: 30pt, {
import cetz.draw: *

smiles-cetz("Nc1ncnc2N(!s{})cnc12", name: "A")
smiles-cetz("Cc1cN(!s{})c(=O)[nH]c1=O", name: "T", origin: (4.9, 0.42))

let hb = (paint: rgb("#3A78C9"), thickness: 1.0pt, dash: "densely-dashed")
let off(anchor, by) = (rel: by, to: anchor)
line(off("A.atom-11", (0.4, -0.15)), off("T.atom-9", (-0.2, 0.06)), stroke: hb)
line(off("A.atom-2", (0.15, 0)), off("T.atom-7", (-0.42, 0.02)), stroke: hb)

// Hydrogen-bond distances.
content((rel: (0.2, 0.2), to: ("A.atom-11", 50%, "T.atom-9")), text(size: 7.5pt, fill: rgb("#3A78C9"))[2.9 Å])
content((rel: (0, 0.28), to: ("A.atom-2", 50%, "T.atom-7")), text(size: 7.5pt, fill: rgb("#3A78C9"))[2.8 Å])

}))
```



The remaining `..opts` are `smiles()` drawing options (`color`, `rotation`, `mirror`, `show-h`, `aromatic`, `charge-style`, `opacity`, `bond-customizations`, …). `fg` and `theme` default to black and light because `auto` foreground resolution is not available inside a raw canvas — pass them explicitly on dark backgrounds.

Use regular CeTZ relative coordinates to offset any connection endpoint: `(rel: (0.12, 0.04), to: "A.atom-0")`. This is the same idea as `atom(..., offset:)` in mechanism arrows, but in raw CeTZ syntax.

> **Note.** Turn on `show-indices: true` while writing anchor references, exactly as for mechanism arrows, to read off the atom indices. Bond-midpoint anchors (`bond-<i>-<j>`) attach to the middle of a bond — useful for marking a reacting or attachment bond.

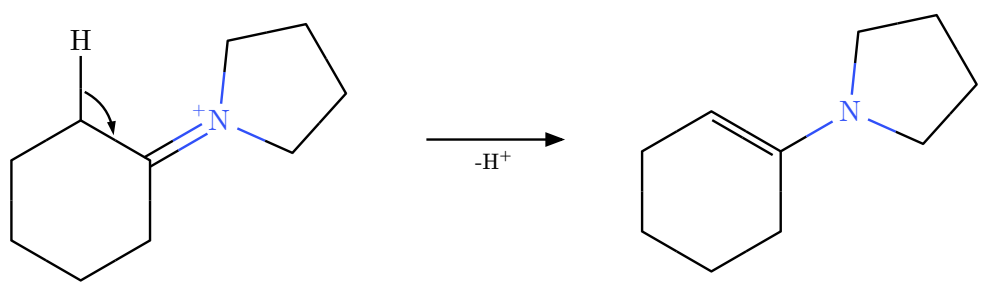
14 Reaction schemes

Three helpers compose molecules, formulas, and arrows into schemes.

14.1 mol()

`mol(spec, ..annotations, label: none, offset: (0,0), ..opts)` is a reaction item. `spec` is either any content (`smiles(...)`, `ce(...)`, text), a SMILES **string**, or a molecule returned by `align-molecules()` (Section 17). Passing a string lets `reaction` render the molecule itself so its atoms become addressable by curly arrows (see Reaction mechanisms). String molecules accept common drawing options such as `scale`, `font-size`, `font`, `bond-stroke`, `color`, `rotation`, `show-h`, `lone-pairs`, `charge-style`, `opacity`, `bond-customizations`, `atom-colors`, `show-indices`, `abbreviate`, and `highlight-overlap`. `offset` nudges the molecule in page coordinates, in bond-length units: positive x moves right and positive y moves up regardless of `reaction(flow:)`. Positional `arrow()` and `highlight()` annotations use local one-molecule references, so `atom(i)` and `bond(i, j)` do not need a species prefix. In mechanism mode, `reaction(scale: ...)` uniformly resizes the complete canvas, including strokes and labels; each `mol(scale: ...)` additionally resizes that molecule. A `mol()` item can also be placed in a `rxn-arrow(above:/below:)` slot to draw a molecule over or under the arrow. In ordinary schemes, an offset also changes the reaction's layout bounds, so `brackets()` encloses the shifted edge.

```typst
#reaction(
  mol(
    "C1({H})C(=[N+]2CCCC2)CCCC1",
    arrow(from: bond(0, 1), to: bond(0, 2)),
  ),
  rxn-arrow(below: [-#ce("H+")]),
  mol("C1=C(N2CCCC2)CCCC1"),
)
```



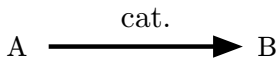
14.2 rxn-arrow()

`rxn-arrow(above: none, below: none, dir: auto, kind: "single", scale: 1.0, color: auto, stroke: auto, label-size: auto)` draws an arrow. `dir` may be `"right"`, `"left"`, `"up"`, `"down"`, or `auto` (the default, following the enclosing `reaction(flow:)`). `kind` may be `"single"`, `"equilibrium"`, `"equilibrium-filled"`, `"dashed"`, or `"wavy"`. `above` and `below` carry reagents and conditions — any content, or a `mol()` item to draw a molecule in the label slot (in mechanism mode a `mol()` there also becomes a referenceable species; see Reaction mechanisms). `scale` uniformly resizes the complete arrow, including its condition labels. `color` defaults to `auto`, inheriting the surrounding text color (so arrows recolor on dark slides). `stroke` sets the shaft width before scaling; `auto` matches the default 0.9pt SMILES bond stroke. `label-size` sets the text size of `above` and `below` content before scaling; `auto` is `0.8em`, so labels follow the surrounding text. Molecules in a slot keep their own size.

Argument	Default	Effect
`above`	`none`	Label above a horizontal arrow, or right of a vertical arrow. Content or a `mol()` item.
`below`	`none`	Label below a horizontal arrow, or left of a vertical arrow. Content or a `mol()` item.
`dir`	`auto`	Arrow direction: `"right"`, `"left"`, `"up"`, `"down"`, or `auto` (follows `reaction(flow:)`).
`kind`	`"single"`	Arrow style: `"single"`, `"equilibrium"`, `"equilibrium-filled"`, `"dashed"`, or `"wavy"`.
`scale`	`1.0`	Uniform arrow scale, including condition labels.
`color`	`auto`	Arrow color; `auto` inherits the surrounding text color.
`stroke`	`auto`	Shaft width before scaling; `auto` matches the default molecule bond stroke.
`label-size`	`auto`	Text size of condition labels before scaling; `auto` is `0.8em` of the surrounding text. Use `1em` to match it, or a length such as `9pt`.

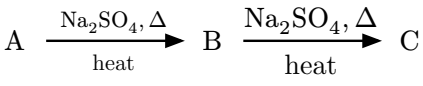
Scale an individual arrow without changing the rest of its reaction. The shaft width, arrowhead, spacing, and condition labels resize together. An explicit `stroke` value is scaled by the same factor.

```typst
#reaction(
  ce("A"),
  rxn-arrow(scale: 1.4, stroke: 1.2pt, above: [cat.]),
  ce("B"),
)
```



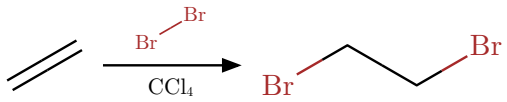
By default, condition labels take 0.8em of the surrounding text. `label-size: 1em` matches the body text, and an absolute length fixes the size regardless of the document.

```typst
#reaction(
  ce("A"),
  rxn-arrow(above: ce("Na2SO4, $Delta$"), below: [heat]),
  ce("B"),
  rxn-arrow(above: ce("Na2SO4, $Delta$"), below: [heat], label-size: 1em),
  ce("C"),
)
```



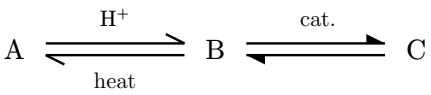
An arrow label slot also accepts a `mol()` item, drawing a molecule above or below the arrow with its own per-molecule options.

```typst
#reaction(
  mol("C=C"),
  rxn-arrow(above: mol("BrBr", scale: 0.7), below: [CCl#sub[4]]),
  mol("BrCCBr"),
)
```



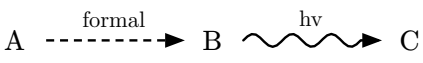
Equilibrium arrows use paired half-heads. The filled variant uses filled half-heads.

```typst
#reaction(
  ce("A"),
  rxn-arrow(kind: "equilibrium", above: ce("H+"), below: [heat]),
  ce("B"),
  rxn-arrow(kind: "equilibrium-filled", above: [cat.]),
  ce("C"),
)
```



A dashed arrow marks a hypothetical or formal transformation; a wavy arrow marks a distorted or non-elementary one.

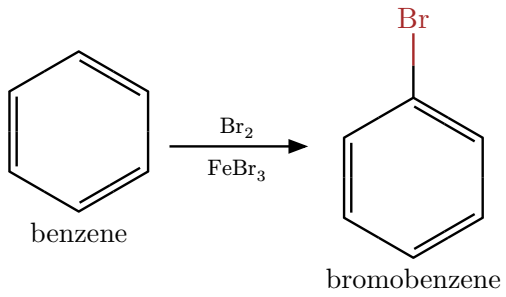
```typst
#reaction(
  ce("A"),
  rxn-arrow(kind: "dashed", above: [formal]),
  ce("B"),
  rxn-arrow(kind: "wavy", above: [hv]),
  ce("C"),
)
```



14.3 `reaction()`

`reaction(gap-h: 1.5em, gap-v: 1.5em, scale: 1.0, breakable: false, show-indices: false, flow: "right", fit: none, highlight-overlap: "merge", ..items)` lays out molecules and arrows left to right. An up or down arrow wraps the scheme onto a new row.

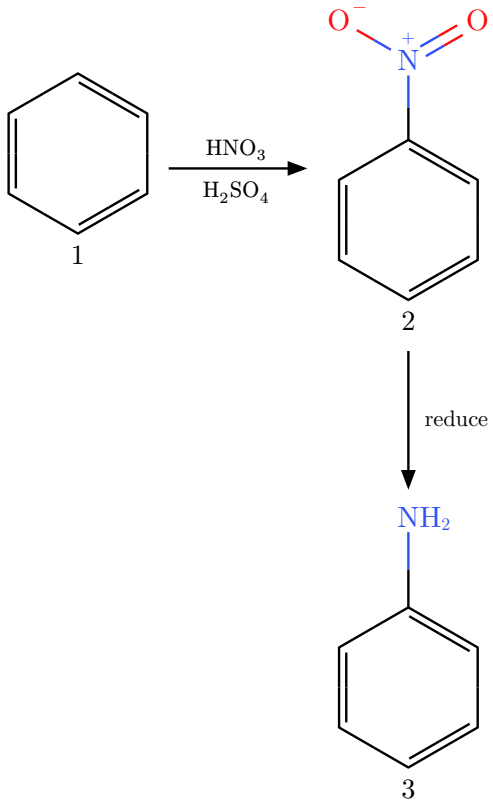
```typst
#reaction(
  mol(smiles("C1=CC=CC=C1"), label: [benzene]),
  rxn-arrow(above: ce("Br2"), below: ce("FeBr3")),
  mol(smiles("BrC1=CC=CC=C1"), label: [bromobenzene]),
)
```



Note. Set `reaction(show-indices: true)` to stamp indices on every string SMILES molecule rendered through `mol("...")` in that reaction. This is useful while authoring large mechanisms with many arrows or highlights. A molecule can still opt out with `mol("...", show-indices: false)`. Molecules already pre-rendered as content, such as `mol(smiles("..."))`, keep their own `smiles()` options.

A wrap-around scheme using a downward arrow:

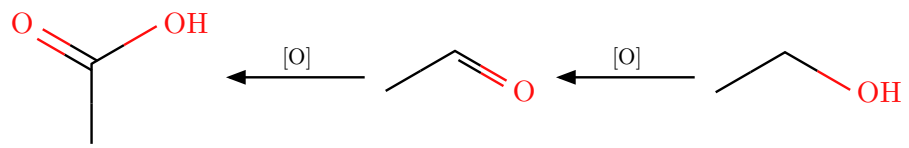
```typst
#reaction(
  mol(smiles("C1=CC=CC=C1"), label: [1]),
  rxn-arrow(above: ce("HNO3"), below: ce("H2SO4")),
  mol(smiles("[O-][N+](=O)C1=CC=CC=C1"), label: [2]),
  rxn-arrow(dir: "down", above: [reduce]),
  mol(smiles("NC1=CC=CC=C1"), label: [3]),
)
```



Flow direction

`flow` sets the writing direction: `"right"` (default), `"left"`, `"up"`, or `"down"`. `"left"` and `"up"` reflect the scheme along the flow axis and flip the arrowheads, so a scheme written in reading order displays right-to-left or bottom-to-top. Ordinary non-arrow items follow the same direction, so `flow: "up"` and `"down"` stack molecules, plus signs, and text vertically even when no `rxn-arrow()` separators are present. This is the natural choice for a branch that grows out of the left or bottom of a `cycle()` — its released product ends up on the side facing the cycle. Arrows created with `rxn-arrow(dir: auto)` (the default) follow the flow; an explicit `dir` is kept.

```typst
#reaction(flow: "left",
  mol(smiles("CCO")),
  rxn-arrow(above: [\[O\]]),
  mol(smiles("CC=O")),
  rxn-arrow(above: [\[O\]]),
  mol(smiles("CC(=O)O")),
)
```

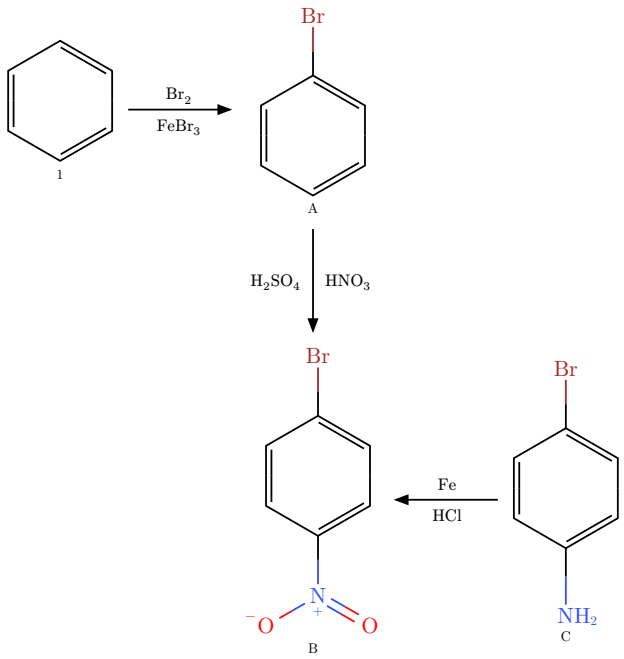


Note. Because `step(out:)` of a `cycle()` accepts any content, a nested `reaction(flow: "left")` (or `"down"`) grows a full sub-reaction out of a released molecule that reads naturally away from the ring, while the outer reaction continues in its own direction. Flow applies to the scheme layout; mechanism mode (curly arrows) is unaffected.

Uniform scale

Pass `scale` to shrink or enlarge the entire scheme uniformly — molecules, arrows, labels, and separators all change together. This is especially useful for multi-step or wrap-around schemes that would otherwise be too wide.

```typst
// same scheme at three different scales
#reaction(
  scale: 0.75,
  mol(smiles("C1=CC=CC=C1"), label: text(size: 7pt)[1]),
  rxn-arrow(above: ce("Br2"), below: ce("FeBr3")),
  mol(smiles("BrC1=CC=CC=C1"), label: text(size: 7pt)[A]),
  rxn-arrow(dir: "down", above: ce("HNO3"), below: ce("H2SO4")),
  mol(smiles("BrC1=CC=C(C=C1)[N+](=O)[O-]"), label: text(size: 7pt)[B]),
  rxn-arrow(dir: "left", above: ce("Fe"), below: ce("HCl")),
  mol(smiles("BrC1=CC=C(C=C1)N"), label: text(size: 7pt)[C]),
)
```



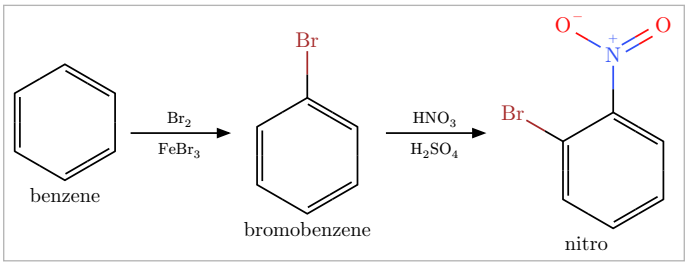
Note. `reaction(scale: ...)` and `smiles(scale: ...)` are independent. The reaction scale is applied on top of however each individual molecule is sized. To resize everything from a single place, use `reaction(scale: ...)` and let each `smiles()` call use its default.

Fitting to the available width

`fit: "width"` shrinks a scheme that is wider than the space it is placed in, so it reads at one uniform size instead of running past a page or card edge. The available width is the width that the surrounding page, block, or grid cell offers. The shrink keeps molecules, arrows, and labels in proportion. A scheme that already fits keeps its natural size. The default, `fit: none`, never resizes a scheme.

Typst

```
#box(width: 9cm, stroke: 0.5pt + luma(180), inset: 4pt,
  reaction(fit: "width",
    mol(smiles("C1=CC=CC=C1"), label: [benzene]),
    rxn-arrow(above: ce("Br2"), below: ce("FeBr3")),
    mol(smiles("BrC1=CC=CC=C1"), label: [bromobenzene]),
    rxn-arrow(above: ce("HNO3"), below: ce("H2SO4")),
    mol(smiles("[O-][N+](=O)C1=CC=CC=C1Br"), label: [nitro]),
  ),
)
```



> **Note.** `fit` is applied after `scale`. `scale` first sets the size of the scheme, and `fit: "width"` then shrinks the result only if it is still wider than the available width. Use `scale` to choose the size on the page and `fit` to guarantee that a scheme never overruns its container.

Page-break behaviour

By default, `reaction` sets `breakable: false`, so the whole scheme moves to the next page as a unit if it does not fit on the current one. This prevents a molecule or vertical branch from stranding on a different page from the rest of the scheme. Set `breakable: true` only for very long schemes that must span pages.

Typst

```
// Default — the scheme always stays on one page:
#reaction( … )                    // breakable: false

// Opt in to page splitting for very long schemes:
#reaction(breakable: true, … )
```

15 Reaction mechanisms

`reaction()` also draws electron-pushing mechanisms: curly arrows that flow from a lone pair, bond, or atom into another bond or atom — within one structure or across separate species. It switches from grid layout to a single shared canvas as soon as any curly `arrow()` or `highlight()` is present. `mol(offset:)` by itself stays in scheme layout and nudges the species inside its grid cell.

Note. On the shared canvas, species are spaced using their true on-screen extent after rotation, mirroring, per-molecule scale, and labels. A molecule that becomes wider or taller once rotated therefore keeps a full gap from its neighbors and from the arrows at any angle, and a `species()` arrow endpoint snaps to the rotated visual edge. `reaction(scale:)` scales the completed canvas uniformly, so molecule bonds, reaction-arrow shafts, curly arrows, labels, bracket strokes, and all lengths stay proportional.

15.1 Referencing atoms

Atoms are addressed by their **writing-order index** (0-based), so the SMILES string is never modified. Inside `smiles()` use single-index references; inside `reaction()` prefix them with the species index `s` — the position of the `mol()`/content item in written order. `mol()` items inside `rxn-arrow(above:/below:)` slots are counted too, at the arrow's position in the sequence (`above` before `below`); the arrows themselves, plain arrow-label content, and annotations are not counted.

Reference	Resolves to
`atom(i)` / `atom(s, i)`	Atom center.
`bond(i, j)` / `bond(s, i, j)`	Midpoint of the visible, label-trimmed bond stroke. Curved arrows attach outside the multiple-bond line facing the curve.
`lp(i)` / `lp(s, i)`	A lone pair on atom i (use `pair: n` to pick one).
`species(k)`	Bounding-box edge of a whole item (e.g. a `ce()` formula).

Note. Every reference accepts an `offset: (dx, dy)` nudge in bond-length units — the escape hatch for fine-tuning an endpoint or pointing at a single-electron dot. Set `show-indices: true` on a `smiles()` / `mol()` to stamp the indices on the diagram while you write the arrows.

Hydrogen atoms declared inside bracket notation (e.g. `[OH-]`, `[NH4+]`) are also addressable. Each H gets the next available index after all heavy atoms, so `atom(1)` in `[OH-]` refers to the hydrogen. With `show-indices`, the heavy-atom and H badges are centered on their rendered label glyphs. Charge marks are excluded from the atom center, so `atom(0)` in `[O-]` points to the O glyph rather than the combined `O-` label.

A displayed stereochemical H has its own atom index. When a new index is needed, it follows the existing atom and virtual-H indices; existing indices remain unchanged. `show-indices: true` reveals it; use `atom(i)` to target the H or `bond(parent, i)` to reference its wedge/hash bond.

Counting species

A species index is an item's position in written order, counting from 0. Every `mol()` counts, and so does every content item, including a plain symbol such as `[+]` placed between reagents. A `mol()` inside an `rxn-arrow(above:)` or `rxn-arrow(below:)` slot counts at the arrow's position, with `above` before `below`. The arrows themselves, the label content of an arrow, and any `arrow()` or `highlight()` do not count.

In the protonation below, ethanol is species 0, the plus sign is species 1, the proton is species 2, and the protonated ethanol is species 3. The arrow from the oxygen lone pair therefore points at `atom(0)` of species 2.

```typst
#reaction(
  mol("CCO", lone-pairs: "dots"),   // species 0
  [+],                              // species 1: a content item counts
  mol("[H+]"),                      // species 2
  rxn-arrow(above: [fast]),         // the arrow and its label do not count
  mol("CC[OH2+]"),                  // species 3
  arrow(from: lp(0, 2), to: atom(2, 0)),
)
```

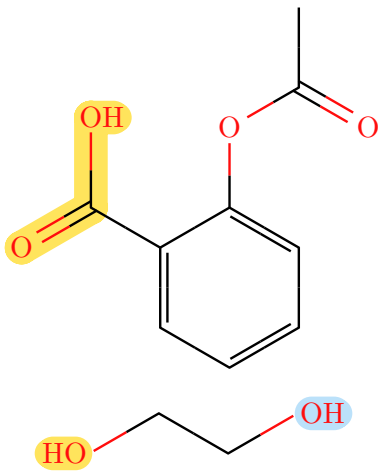


Note. A species index that does not exist, or that names a content item where atoms are needed, stops compilation with a list of every index and its item. If `atom(5, 0)` replaced `atom(2, 0)` in the reaction above, the message would give the valid range `0` through `3` and list `0 = mol("CCO")`, `1 = content [+]`, `2 = mol("[H+]")`, and `3 = mol("CC[OH2+]")`, so the reference can be corrected at once.

15.2 Automatic substructure highlighting

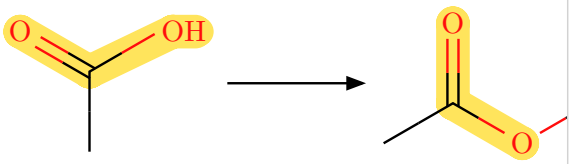
Pass `highlight-smarts` to select atoms and bonds by chemistry. `highlight-groups` offers named groups without writing SMARTS. Both accept a string, a request dictionary, or a tuple mixing both, and shade every distinct match with the existing highlight style.

```typst
#smiles("CC(=O)OC1=CC=CC=C1C(=O)O",
  highlight-smarts: "C(=O)[OX2H1]")
#smiles("OCCO", highlight-groups:
"alcohol",
  highlight-colors: (rgb("#FFE45C"),
  rgb("#BBE1FA")))
```



These options also work with `smiles-inline()`, `smiles-cetz()`, and `mol()`, including schemes with curly mechanism arrows.

```
#reaction(
  mol("CC(=O)O", highlight-groups: "carboxylic acid"),
  rxn-arrow(),
  mol("CC(=O)OC", highlight-groups: "ester"),
)
```



Matches are sorted by atom indices, then query bonds. Automatic highlights set `include-atoms: true`, so bond capsules join at their endpoint atoms into a continuous highlight. Each request can override this: use `highlight-smarts: (pattern: "c1ccccc1", include-atoms: false)` or `highlight-groups: (group: "carbonyl", include-atoms: false)`. This draws trimmed bond capsules without endpoint atom disks. Standalone matched atoms (including single-atom groups) are still shaded. Strings and dictionaries may be mixed in a tuple; the override applies to all matches of that request. A request also accepts `include-hydrogens` (default `auto`) for the displayed H labels of selected heteroatoms; see the `highlight()` section below for its values. Symmetric mappings of the same atoms and bonds count once; overlapping matches remain. The six-color palette cycles across SMARTS requests, then group requests. Set `highlight-colors: (yellow, blue)` to customize it; a single-color tuple gives every match that color. Shared regions take the later highlight color; manual `highlight()` annotations draw afterward. Only bonds specified by the query are shaded, including ring closures; recursive context is not shaded.

No match produces a diagnostic naming the pattern/group and molecule. Use `highlight-unmatched: "ignore"` to allow absent groups explicitly. Invalid or unsupported patterns still produce an error. For inspection, `substructure-matches(smiles-str, pattern)` returns an array of dictionaries with `atoms` (writing-order indices) and `bonds` (endpoint pairs), or `()` when absent. For example, acetic acid with `C(=O)[OX2H1]` gives `((atoms: (1, 2, 3), bonds: ((1, 2), (1, 3))),)`.

Named functional groups

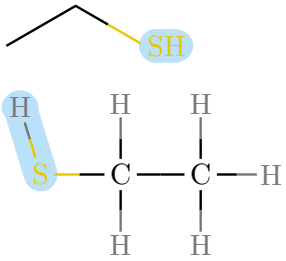
Names accept spaces or hyphens and ignore letter case. The exported `functional-groups` dictionary contains the exact SMARTS definitions.

Name	Selection
`carboxylic-acid` / `carboxylate`	Carbonyl carbon and both oxygens; neutral OH / negatively charged O.
`alcohol` / `phenol`	Neutral OH attached to saturated / aromatic carbon.
`amine`	Neutral primary, secondary, or tertiary amine N; excludes amides, sulfonamides, amidines, guanidines, and cyanamides.
`ester`	Carboxylic ester C(=O)O; includes formates, excludes anhydrides, carbonates, and carbamates.
`amide`	Each local O=C–N motif, including lactams, ureas, and carbamates. Urea has two overlapping matches.
`carbonyl`	C=O, including aromatic carbonyl atoms and carbonyls in acids, esters, and amides.
`aldehyde` / `ketone`	C=O with aldehyde H (including formaldehyde) / two carbon neighbors.

Name	Selection
`nitrile`	C triple-bonded to terminal N, attached to carbon or H; excludes cyanamide.
`ether` / `thiol`	Neutral ether O excluding esters / neutral SH attached to carbon.
`nitro`	The charged N(=O)O group attached directly to carbon; excludes nitrate esters.
`alkene` / `alkyne`	Non-aromatic C=C / C triple-bonded to C.

These are structural definitions; matching does not normalize protonation or tautomers. Alcohol excludes phenol and carboxylic-acid OH. Recursive predicates check neighboring chemistry without adding those neighbors to the highlight. Alcohol, phenol, amine, and thiol shade their O, N, or S atom together with displayed attached H, including `SH`, `OH`, and `NH₂` labels. Skeleton mode shades the corresponding H atoms and bonds. Carboxylic-acid OH, amide NH, and aldehyde H are included likewise. Omitted hydrogens stay hidden; bond-only requests leave endpoint atoms and their H unshaded. Ether selects O. Named groups keep this curated list under `include-hydrogens: auto` `include-hydrogens: false` removes these H, and `include-hydrogens: true` shades the H of every selected heteroatom instead. Carboxylic acid and carboxylate exclude carbonic and carbamic acids.

`substructure-matches` returns only query atom/bond indices; named-group highlights add the displayed H fragments during drawing. Raw SMARTS highlights follow their query selection.



The test corpus checks 63 PubChem compounds and all 16 groups against reviewed chemical counts and independent RDKit atom/bond selections. It also checks both `include-atoms` settings and explicit uppercase Kekulé notation.

SMARTS subset and aromaticity

Supported syntax includes element/aromatic symbols, wildcard `*`, atomic numbers (`[#6]`), hydrogen count `H`, graph degree `D`, total connectivity `X` including attached H, formal charges (`+`, `-`, `+0`), ring membership `R` / `R0`, branches, ring closures (digits and `%nn`), and dot-separated components. Atom predicates support negation `!`, high-precedence AND (`&` or juxtaposition), OR `,`, low-precedence AND `;`, and anchored recursive `$(...)`. Bonds support `-`, `=`, `#`, `:`, and any bond `~`. An omitted bond accepts single or aromatic bonds.

Hydrogens and charges reuse the molecule graph, including bracket and folded terminal hydrogens. `[H]` selects a retained hydrogen atom; `[OH1]` counts attached hydrogens. Display-only H fragments are not independent query atoms. Abbreviations have no inferred composition; use `*` to select them.

Lowercase input retains aromatic flags after single/double assignment, so `c:c` matches both Kekulé and circle drawing styles. Uppercase `C1=CC=CC=C1` has no perceived aromatic flags. Use lowercase input for aromatic predicates, or `[#6]~[#6]` to match carbon bonds in either notation. An explicit biphenyl `-` linker remains non-aromatic.

PubChem commonly exports uppercase Kekulé strings. Use lowercase aromatic notation when classifying aromatic molecules with named groups: uppercase phenol will not match `phenol`, and its explicit C=C bonds can match `alkene`. Aromatic nitrogen classification also depends on this notation.

This is a documented subset, not full RDKit/Daylight SMARTS. Stereo, isotope, atom-map, implicit-H (`h`), valence/hybridization, ring-size/count, and bond logical predicates produce explicit unsupported-syntax errors. Limits are 4096 pattern bytes, 64 atoms per query, 16 nesting levels, 1,000,000 search steps, and 4096 distinct matches. Exceeding a limit gives an error without partial highlight results.

15.3 `arrow()` and `highlight()`

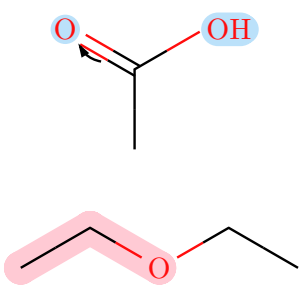
`arrow(from:, to:, label: none, color: black, stroke: auto, bend: "left", angle: 15deg, half: false, heads: "end", head-length: 0.11, head-width: 0.07, style: "solid")` draws a curly arrow between two references. `bend` is `"left"`, `"right"`, or `none`; `angle` sets how strongly it bows; `half: true` draws fishhook (single-electron) heads. `heads` selects which ends carry an arrowhead — `"end"` (default), `"both"`, or `"none"` — and `style` the shaft: `"solid"` (default), `"dashed"`, or `"wavy"`; both combine freely with `bend`. The compact triangle tip is sized in bond-length units by `head-length` (along the shaft) and `head-width` (base); they apply to every drawn head and scale with the drawing. `highlight(ref, fill:)` shades an atom (disk) or bond (capsule) behind the structure. Pass an array of references to shade several atoms or bonds with one call. Bond highlights also shade their endpoint atoms by default, so bonded highlights join into one continuous region; set `include-atoms: false` for bond-only capsules trimmed away from atom centers. `include-hydrogens` (default `auto`) also shades the displayed H labels of selected heteroatoms, such as the H of an N–H or O–H, so the highlight covers them. `include-hydrogens: false` leaves those labels unshaded. Carbon H labels shown with `show-h` are not shaded by a manual highlight. `radius` sets the half-width of the highlighted band in bond-length units: it sets the disk radius of atom highlights and endpoint disks, the half-width of bond capsules, and the half-height of label capsules. With the default `radius: auto`, a bond band has its usual width and its endpoint disks match that width; atom highlights keep the atom size. Abbreviated group labels such as `{PPh3}` are highlighted as measured label-width capsules rather than atom-sized disks.

> **Note.** **Migration:** `highlight()` now shades bond endpoints by default; pass `include-atoms: false` for bond-only capsules. It also shades the displayed H of selected heteroatoms; pass `include-hydrogens: false` to keep those H unshaded.

`stroke` sets the unscaled shaft width. Its `auto` default matches the molecule's effective bond stroke, including an explicit `bond-stroke`. Explicit arrow strokes scale with `smiles(scale:)` or `reaction(scale:)`.

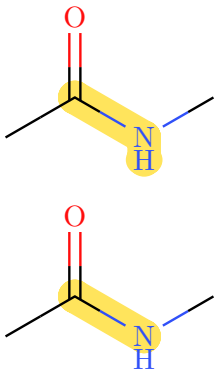
Typst

```
#smiles(
  "CC(=O)O",
  highlight((atom(2), atom(3)), fill: rgb("#BBE1FA")),
  arrow(from: bond(1, 2), to: atom(2)),
)
#h(1em)
#smiles(
  "CCOCC",
  highlight((bond(0, 1), bond(1, 2)), fill: rgb("#FFCAD4")),
)
```



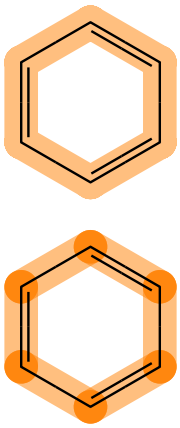
The displayed H of a selected heteroatom joins the highlight, so the amide N–H stays inside the band. `include-hydrogens: false` leaves it unshaded.

```typst
#smiles(
  "CC(=O)NC",
  highlight(bond(1, 3), fill: rgb("#FFE45C")),
)
#smiles(
  "CC(=O)NC",
  highlight(bond(1, 3), fill: rgb("#FFE45C"), include-hydrogens: false),
)
```



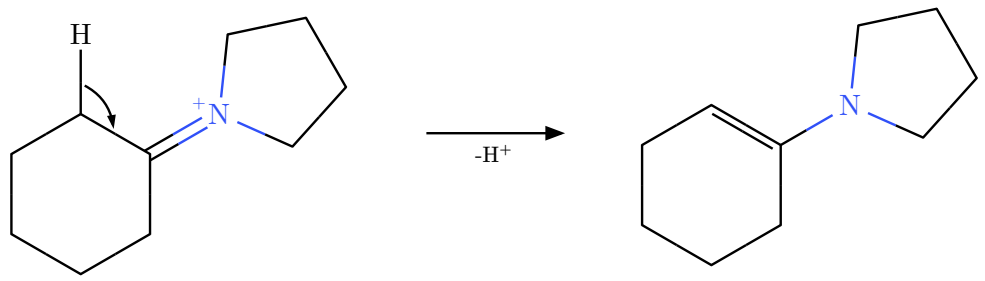
Translucent highlights, such as `rgb("#FF8000").transparentize(50%)`, are painted as one shape per fill color by default. A bond capsule and the atom disks at its ends keep the same tint where they meet, and bonds that share an atom do not darken there. `highlight-overlap: "stack"` paints every piece on its own, so the overlaps are darker again. The same option is accepted by `smiles()`, `mol()`, and `reaction()`.

```typst
#smiles(
  "C1=CC=CC=C1",
  highlight(
    (bond(0, 1), bond(1, 2), bond(2, 3), bond(3, 4), bond(4, 5), bond(5, 0)),
    fill: rgb("#FF8000").transparentize(50%),
  ),
)
#smiles(
  "C1=CC=CC=C1",
  highlight(
    (bond(0, 1), bond(1, 2), bond(2, 3), bond(3, 4), bond(4, 5), bond(5, 0)),
    fill: rgb("#FF8000").transparentize(50%),
  ),
  highlight-overlap: "stack",
)
```



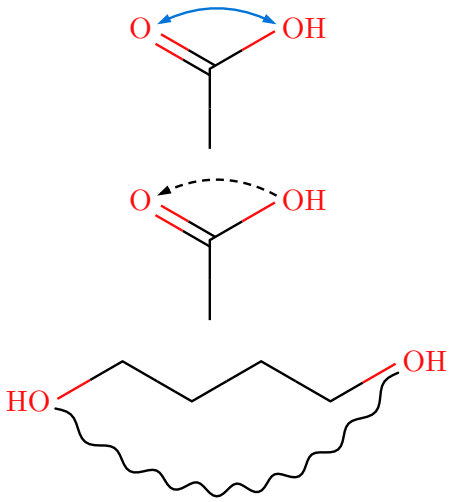
Put mechanism annotations positionally inside `mol()` when every reference belongs to that molecule. The short forms `atom(i)`, `bond(i, j)`, and `lp(i)` are scoped to the containing molecule, independent of its species index in the outer reaction.

```typst
#reaction(
  mol(
    "C1({H})C(=[N+]2CCCC2)CCCC1",
    arrow(from: bond(0, 1), to: bond(0, 2)),
  ),
  rxn-arrow(below: [-#ce("H+")]),
  mol("C1=C(N2CCCC2)CCCC1"),
)
```



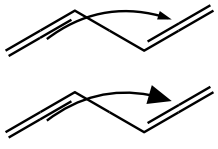
`heads` and `style` tune the arrow shape — for instance a double-headed dashed arrow for an interaction, or a wavy arrow for a photochemical or homolytic step.

```typst
#smiles("CC(=O)O",
  arrow(from: atom(3), to: atom(2), heads:
  "both", color: blue, bend : "right"))
#smiles("CC(=O)O",
  arrow(from: atom(3), to: atom(2), style:
  "dashed", bend : "right"))
#smiles("OCCCCO",
  arrow(from: atom(0), to: atom(5), style:
  "wavy", heads: "none", half: true,
  bend : "right", angle : 30deg))
```



The tip defaults to a small, thin triangle so it stays clear of nearby bonds. Raise `head-length` and `head-width` for a bolder head.

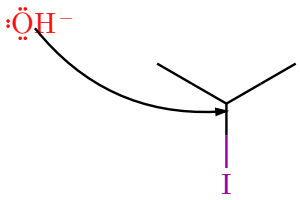
```typst
#smiles("C=CC=C",
  arrow(from: bond(0, 1), to: bond(2, 3)))
#smiles("C=CC=C",
  arrow(from: bond(0, 1), to: bond(2, 3),
  head-length: 0.24, head-width: 0.2))
```



15.4 A mechanism across species

Hydroxide attacks the central carbon; the C–I bond breaks toward iodide. The nucleophile is offset so the curly arrow reads cleanly.

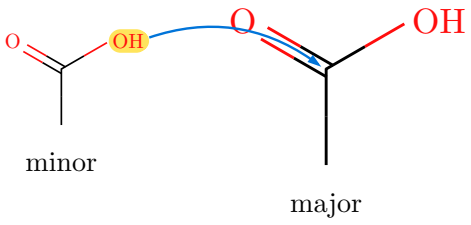
```typst
#reaction(
  mol("[OH-]", lone-pairs: "dots", offset: (0, 1)),
  mol("C(I)(C)C"),
  arrow(from: lp(0, 0, offset:(0.1, -0.2)), to: atom(1, 0, offset : (0.1, -0.1)),
        bend: "right", color : black),
)
```



15.5 Per-molecule scale in mechanisms

In mechanism mode `reaction(scale:)` sets the shared canvas scale and each `mol(scale:)` multiplies it for its own species. Curly arrows and highlights land on the scaled coordinates.

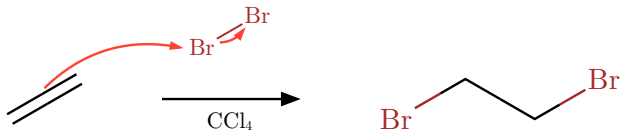
```typst
#reaction(
  mol("CC(=O)O", scale: 0.7, label: [minor]),
  mol("CC(=O)O", scale: 1.2, label: [major]),
  highlight(atom(0, 3)),
  arrow(from: atom(0, 3), to: atom(1, 1), color: blue),
)
```



15.6 Referencing molecules above an arrow

A `mol()` placed in a `rxn-arrow(above:/below:)` slot is lifted onto the shared canvas as a species of its own, so its atoms take part in the mechanism like any other species.

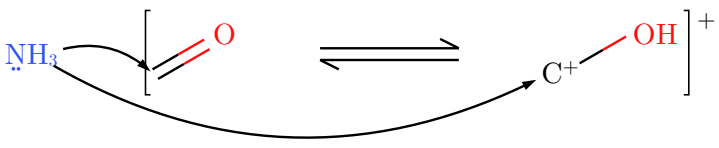
```typst
#reaction(
  mol("C=C"),
  rxn-arrow(above: mol("BrBr", scale: 0.8), below: [CCl#sub[4]]),
  mol("BrCCBr"),
  arrow(from: bond(0, 0, 1), to: atom(1, 0), color: red),
  arrow(from: atom(1, 0), to: atom(1, 1), color: red, bend: "right"),
)
```



15.7 `brackets()`

`brackets(body, sup: none, sub: none)` encloses a single content value. Inside `reaction()`, `brackets(..items, sup:, sub:)` also accepts the same list of reaction items as its parent. Those items stay on the parent's shared canvas: their ordinary written-order species indices remain available to arrows inside or outside the brackets. The parent `reaction(scale:)` scales the contents and bracket together.

```typst
#reaction(
  mol("N", lone-pairs: "dots"),
  brackets(
    mol("C=O"),
    rxn-arrow(kind: "equilibrium"),
    mol("{C^+}O"),
    sup: [+],
  ),
  arrow(from: atom(0, 0), to: atom(1, 0)),
  arrow(from: atom(0, 0), to: atom(2, 0), bend: "right"),
)
```



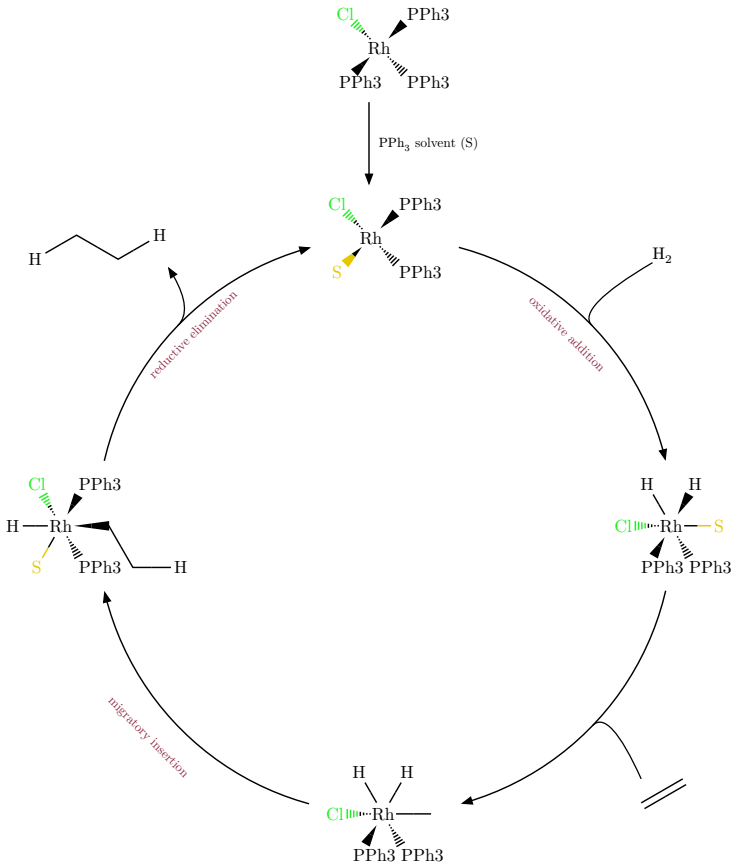
16 Catalytic cycles: `cycle()`

`cycle(..items, radius:, start:, clockwise:, scale:, reagent-bend:, ...)` arranges species on a circle with arc arrows between them — the standard depiction of a catalytic cycle. Items alternate species and step()s, just as `reaction()` alternates molecules and arrows, but the sequence closes into a ring (the last step returns to the first species). Species are mol() items or any content; SMILES strings are rendered by `smiles()`.

`step(label:, into:, out:, bend:, merge:, rotation:, label-offset:, into-offset:, out-offset:)` describes the arc that follows a species: `label` names the transformation, `into` adds a reagent merging into the arc from outside the ring, and `out` a product leaving it. Those side arrows are bookkeeping arrows for material entering or leaving the catalytic manifold, not electron-pushing arrows. `merge: true` draws a side arrow tangent to the arc so it visually fuses with the main cycle arrow. `rotation` rotates the step label: `"straight"` keeps it horizontal, `"auto"` follows the step's circle angle while keeping text upright, and an explicit angle is used as given. The three `*-offset` arguments nudge the label and the into/out reagents like a `mol` offset.

Typst

```
#align(center)[
#reaction(
  scale : 0.6,
  flow : "down",
  mol(smiles("{Rh}(!w{>PPh3})(!h{>PPh3})(!hCl)(!w{>PPh3})"), offset: (0.5, 0)),
  rxn-arrow(above : [#ce("PPh3") solvent (S)]),
  cycle(
    radius : 6.0,
    mol(smiles("{Rh}(!w{S | S})(!h{>PPh3})(!hCl)(!w{>PPh3})")),
    step(label : [#text(size : 8pt)[oxidative addition]], label-offset : (0.3, 0.5),
    into : [#ce("H2")], rotation : "auto", merge : true),
    mol(smiles("{Rh}(!w{>PPh3})(!h{>PPh3})(!hCl)({S | S})({H})(!w{H})")),
    step(into : [#smiles("C=C")], merge : true),
    mol(smiles("{Rh}(!w{>PPh3})(!h{>PPh3})(!hCl)({})({H})({H})")),
    step(label : [#text(size : 8pt)[migratory insertion]], label-offset: (0, 0.5),
    rotation : "auto"),
    mol(smiles("{Rh}(!w{>PPh3})(!h{>PPh3})(!hCl)({S | S})({H})(!wCC{H})", rotation :
    -90deg)),
    step(label : [#text(size : 8pt)[reductive elimination]], label-offset:  (-0.5,
    0.5), out : [#smiles("{H}CC{H}")], merge:true, rotation : "auto")
  )
)
]
```

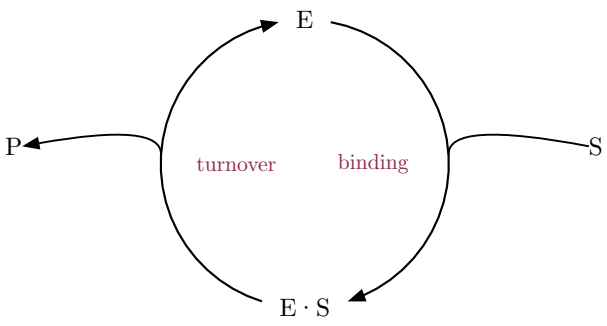


Because `step(out:)` accepts any content, passing a `reaction()` there grows a full linear branch out of a released molecule — the released product becomes the first species of a downstream sequence. Reagents (both `into` and `out`) are offset by their own measured size, so even a wide nested reaction clears the ring. Incoming content is attached on the side facing the cycle so the side arrow starts at the last upstream item; outgoing content uses the same side so the leaving arrow lands at the first downstream item instead of the center of the whole branch.

> **Note.** `radius` defaults to `auto`, which fits the species without overlap; set it explicitly for a tighter or looser ring. `start` is the angle of the first species (`90deg` is the top) and `clockwise` the travel direction. `scale` uniformly resizes species, arrows, and labels together. `arc-gap` tunes how close the arc arrows sit to the species (in bond-length units beyond each molecule's radius; smaller or even negative brings them closer). `reagent-bend` sets the default curvature of into/out side arrows; `step(bend:)` overrides one step, and `bend: 0` makes that side arrow nearly radial. A one-species `cycle()` draws that species alone.

With `merge: true` the into/out arrows fuse tangentially with the main cycle arrows, and `arc-gap` pulls the arrows closer to the species:

```typst
#cycle(
  scale: 0.9,
  arc-gap: 0.0,
  mol(box(inset: 2pt)[E]),
  step(label: [binding], into: [S], merge: true),
  mol(box(inset: 2pt)[E·S]),
  step(label: [turnover], out: [P], merge: true),
)
```



17 Molecule series

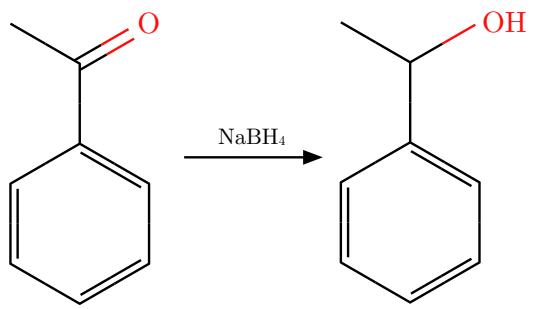
Comparison figures — a starting material and its product, or a numbered compound series — read best when the shared core keeps one orientation and every molecule is drawn at the same scale. `align-molecules()` orients a series onto a common scaffold, and `molecule-grid()` lays out a series at one shared bond length.

17.1 Aligning a series: `align-molecules()`

`align-molecules(molecules, scaffold:, atoms:, reference:, rotation:, mirror:, allow-reflection:)` takes an array of SMILES strings and returns one **aligned molecule** per input. Each molecule is turned, optionally mirrored, and shifted so its scaffold lies over the reference molecule's scaffold. Pass an aligned molecule anywhere a SMILES string is accepted: `smiles()`, `smiles-inline()`, `smiles-cetz()`, `mol()`, `reaction()`, `cycle()`, and `molecule-grid()`.

```
#let aligned = align-molecules(
  ("CC(=O)c1ccccc1", "CC(O)c1ccccc1"),
  scaffold: "c1ccccc1",
)
#reaction(
  mol(aligned.at(0)),
  rxn-arrow(above: [NaBH#sub[4]]),
  mol(aligned.at(1)),
)
```

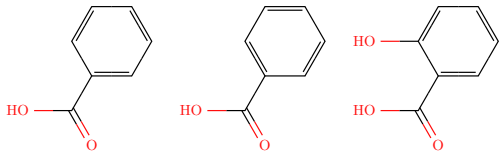
Typst



Argument	Default	Meaning
`molecules`	—	Array of two or more SMILES strings.
`scaffold`	`none`	SMARTS pattern (Section 15) found once in every molecule. Plain SMILES such as `"c1ccccc1"` work as patterns.
`atoms`	`none`	One entry per molecule: `auto`, or an array of atom indices (as shown by `show-indices: true`). With a scaffold, the array selects which occurrence to use; without one, the arrays correspond atom by atom across the series.
`reference`	`0`	Index of the molecule whose own layout sets the orientation.
`rotation`	`0deg`	Rotation of the reference, and so of the whole series.
`mirror`	`none`	`"horizontal"` or `"vertical"` page-axis reflection of the whole series.
`allow-reflection`	`true`	Let a molecule be mirrored when that overlays its scaffold better than any rotation.

The same compound written in different atom orders is laid out in different orientations; aligned on a shared scaffold, the series reads at a glance. `rotation` and `mirror` turn the whole series at once.

```typst
#let acids = align-molecules(
  ("c1ccccc1C(=O)O", "O=C(O)c1ccccc1",
   "OC(=O)c1ccccc1O"),
  scaffold: "c1ccccc1C(=O)O",
  rotation: 90deg,
)
#grid(
  columns: 3, gutter: 1em,
  ..acids.map(acid => smiles(acid, scale: 0.5)),
)
```

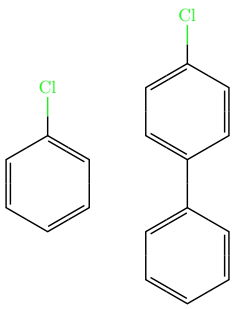


Symmetric scaffolds and several occurrences

A symmetric scaffold such as benzene fits the reference in several ways. `align-molecules()` picks the correspondence that puts substituted scaffold atoms where the reference has its substituents — preferring substituents of the same element — then the closest overlay, then a rotation over a reflection. The choice is deterministic.

When the scaffold occurs at places that are interchangeable by symmetry, such as the two rings of biphenyl or the methyls of an isobutyl group, the best fitting one is used. When it occurs at chemically distinct places, such as either ring of 4-chlorobiphenyl, alignment stops with an error that lists the candidate atoms; choose one by passing its atoms, in any order, in `atoms`:

```typst
#let biphenyls = align-molecules(
  ("Clc1ccccc1", "c1ccccc1-c1ccc(Cl)cc1"),
  scaffold: "c1ccccc1",
  atoms: (auto, (6, 7, 8, 9, 11, 12)),
)
#grid(
  columns: 2, gutter: 2em, align: horizon,
  ..biphenyls.map(biphenyl =>
  smiles(biphenyl, scale: 0.6)),
)
```



With a scaffold, an atom list must cover exactly one place where the scaffold occurs, and the substituent rules above still decide how its atoms correspond. For a fixed atom-by-atom correspondence, omit the scaffold and give every molecule an atom list of the same length: atom `k` of each list is placed over atom `k` of the reference's list. At least two corresponding atoms are needed.

What alignment changes

Alignment rotates, optionally mirrors, and translates each molecule's own layout; it does not redraw the scaffold. Ring cores are drawn identically by the layout engine and therefore coincide, but a chain scaffold drawn with a different zigzag in two molecules keeps a residual mismatch. Each aligned molecule reports it as `deviation`: the largest distance, in bond lengths, between a scaffold atom and its reference atom. Its `offset` field gives the translation before the final rotation. Its `atoms` field lists the molecule's scaffold atoms in scaffold order, ready for `highlight()` references.

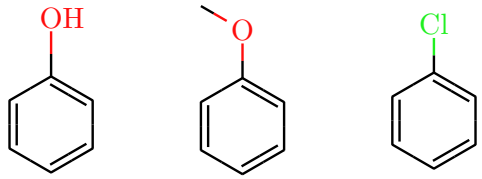
A mirrored molecule exchanges wedges and hashes, so the depicted stereochemistry is unchanged. Because an aligned molecule already carries its orientation, `rotation` and `mirror` cannot be passed again with it; turn the whole series through `align-molecules()` instead. `show-h: "skeleton"` redraws carbon

chains and is not available for aligned molecules, and neither is `abbreviate`, which lays out a contracted structure that the alignment was not computed for.

17.2 Compound grids: `molecule-grid()`

`molecule-grid(columns:, scale:, bond-length:, sizing:, scaffold:, ..items)` arranges SMILES strings, aligned molecules, and `mol()` items in equal-width columns. Every molecule shares one bond length and is centered in its cell, so a small molecule is never enlarged to fill its space. Captions come from `mol(label: ...)` and start on one line across each row.

```typst
#molecule-grid(
  columns: 3,
  bond-length: 0.6,
  mol("Oc1ccccc1", label: [*1* Phenol]),
  mol("COc1ccccc1", label: [*2* Anisole]),
  mol("Clc1ccccc1", label: [*3* PhCl]),
)
```

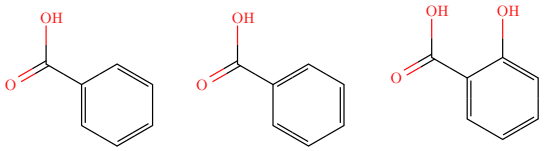


1 Phenol **2** Anisole **3** PhCl

Argument	Default	Meaning
`columns`	`auto`	Number of equal-width columns; `auto` uses one per molecule, up to four.
`scale`	`1.0`	Shared scale of bonds, labels, and strokes.
`bond-length`	`none`	Shared bond length (1.0 = 30 pt), overriding the one implied by `scale`.
`sizing`	`"fixed"`	`"fixed"` draws at the given scale; `"fit"` scales the whole series by one factor so the widest molecule fills its column.
`scaffold`	`none`	Align every molecule onto the first one's occurrence of this SMARTS pattern.
`column-gutter`	`1.5em`	Space between columns.
`row-gutter`	`1.5em`	Space between rows.
`label-gap`	`0.6em`	Space between a molecule and its caption.
`breakable`	`true`	Allow page breaks between rows; `false` keeps the grid on one page.

With `sizing: "fit"` the series keeps its relative sizes while filling the available width, and `scaffold:` aligns it like `align-molecules()` with the first molecule as reference.

```typst
#molecule-grid(
  columns: 3,
  sizing: "fit",
  scaffold: "c1ccccc1C(=O)O",
  mol("c1ccccc1C(=O)O", label: [Benzoic]),
  mol("O=C(O)c1ccccc1", label: [Same]),
  mol("OC(=O)c1ccccc1O", label:
  [Salicylic]),
)
```



Benzoic Same Salicylic

Note. Drawing options such as `show-h`, `color`, or highlights go on each `mol()` item. A per-item `scale` or `bond-length` would break the shared scale and is rejected; set it on the grid. With `sizing:`

> `"fixed"` a molecule wider than its column stops with an error that suggests a smaller scale, fewer columns, or `sizing: "fit"`. Page breaks fall only between rows, so a molecule never separates from its caption.

18 Project-wide defaults

Typst functions support partial application with `.with()`. Calling `smiles.with(...)` returns a new function with the given arguments pre-filled. Rebind the name in your preamble and every subsequent `#smiles(...)` call uses those defaults — including inside `#reaction()`, because the content is evaluated before `reaction` sees it.

Any argument can be pre-filled this way: `bond-length`, `font`, `scale`, `font-size`, `atom-colors`, or any other `smiles` parameter.

Typst

```typst
// ── preamble ──────────────────────────────────────────────────────
#import "@preview/typed-smiles:0.13.0": *

#let smiles = smiles.with(
  bond-length: 0.9,
  font:        "Libertinus Serif",
  atom-colors: (O: rgb("#8B4513"), N: rgb("#008080")),
)

// ── anywhere in the document ──────────────────────────────────────
#smiles("CC(N)C(=O)O")   // uses bond-length 0.9, Libertinus, custom O/N
#reaction(
  smiles("CCO"),
  rxn-arrow(),
  smiles("CC(=O)O"),     // same custom smiles — preamble applies here too
)
```

Note. Any argument passed directly at the call site still overrides the `.with()` default for that call alone.

19 Limitations

- R/S and E/Z descriptors are not available; `@`/`@@` and directional bonds control the depiction only.
- Trigonal-bipyramidal (`@TB1`–`@TB20`), octahedral (`@OH1`–`@OH30`), and allene (`@AL`) stereochemistry is not drawn; by default it is reported as an error (see Section 7.4).
- Trans double bonds in rings of fewer than nine atoms, and some chelate rings around square-planar centers, cannot be laid out with the written geometry and are reported the same way.
- Reaction SMILES (`A>>B`), dative-bond arrows (`->`), and `%(nnn)` ring numbers are not read; draw reactions with `reaction()`.
- Ring stereochemistry between adjacent centers can need a manual adjustment (try `rotation`, or the manual `!w` and `!h` wedges).
- Some three-dimensional skeletons, such as triptycene, cannot be drawn flat without one ring overlapping another.
- `align-molecules()` rotates and mirrors existing layouts; it does not redraw a scaffold to match the reference. Chain scaffolds drawn with different zigzags keep a residual `deviation` (Section 17).

20 Molecule library

Every name available as `molecules.<name>`, grouped by category: small molecules and solvents, aromatics and heterocycles, nucleobases, amino acids, sugars, acids, drugs and natural products, and laboratory reagents.

water



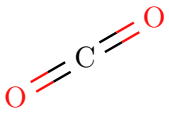
methane



ammonia



carbon-dioxide



hydrogen-chloride



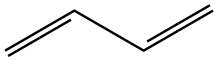
ethylene



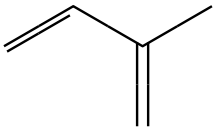
acetylene



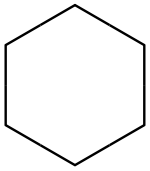
butadiene



isoprene



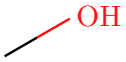
cyclohexane



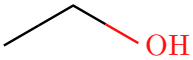
hexane



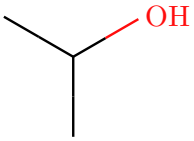
methanol



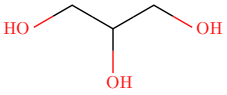
ethanol



isopropanol



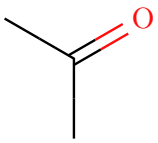
glycerol



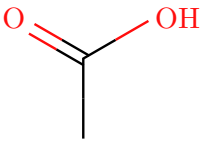
formaldehyde



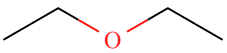
acetone



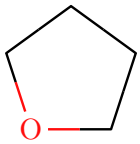
acetic-acid



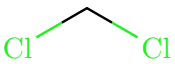
diethyl-ether



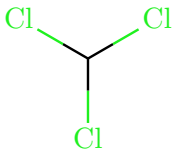
tetrahydrofuran



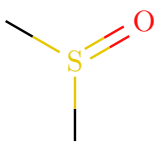
dichloromethane



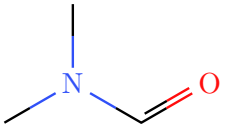
chloroform



dimethyl-sulfoxide



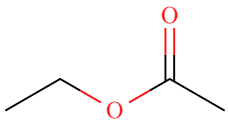
dimethylformamide



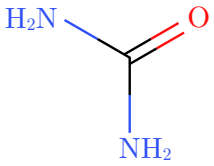
acetonitrile



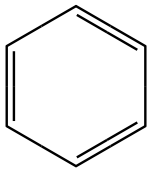
ethyl-acetate



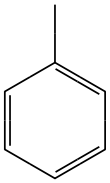
urea



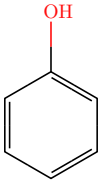
benzene



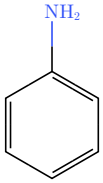
toluene



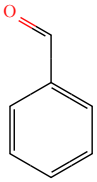
phenol



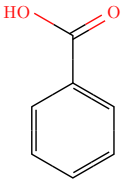
aniline



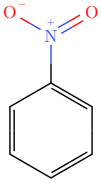
benzaldehyde



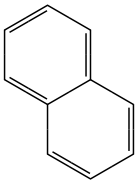
benzoic-acid



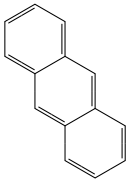
nitrobenzene



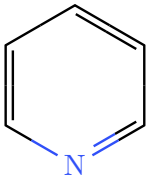
naphthalene



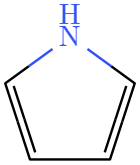
anthracene



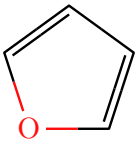
pyridine



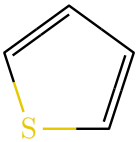
pyrrole



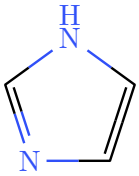
furan



thiophene



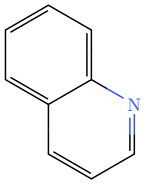
imidazole



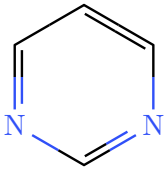
indole



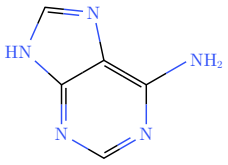
quinoline



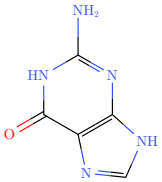
pyrimidine



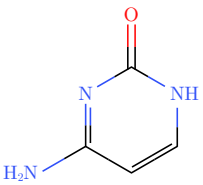
adenine



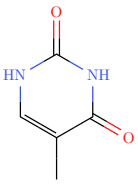
guanine



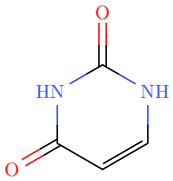
cytosine



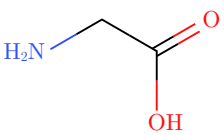
thymine



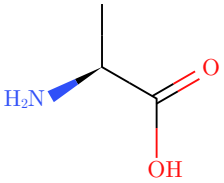
uracil



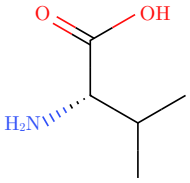
glycine



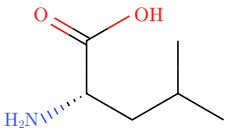
alanine



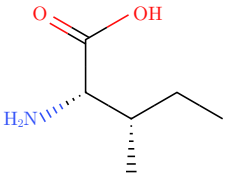
valine



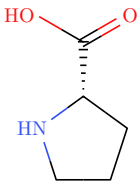
leucine



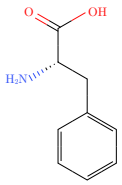
isoleucine



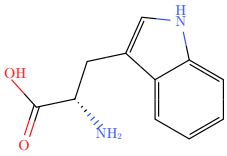
proline



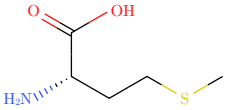
phenylalanine



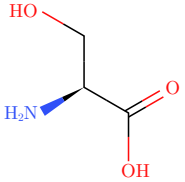
tryptophan



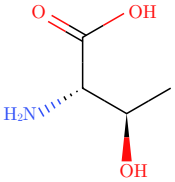
methionine



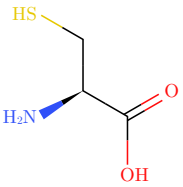
serine



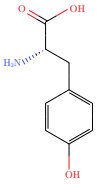
threonine



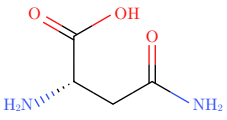
cysteine



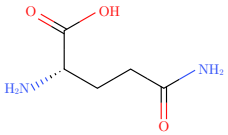
tyrosine



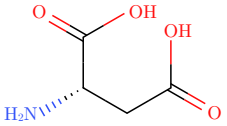
asparagine



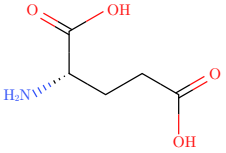
glutamine



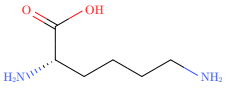
aspartic-acid



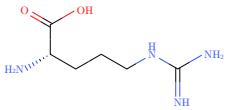
glutamic-acid



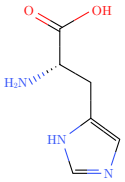
lysine



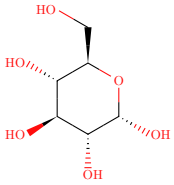
arginine



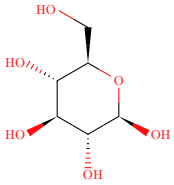
histidine



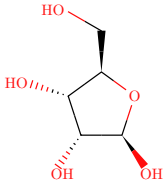
alpha-d-glucose



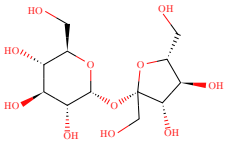
beta-d-glucose



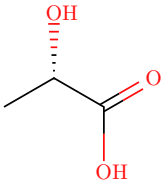
beta-d-ribose



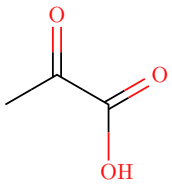
sucrose



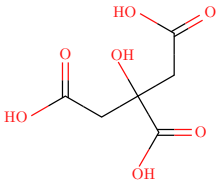
lactic-acid



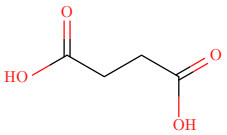
pyruvic-acid



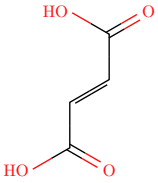
citric-acid



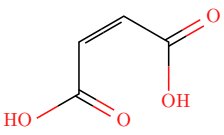
succinic-acid



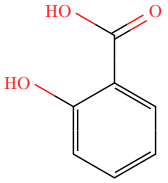
fumaric-acid



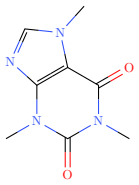
maleic-acid



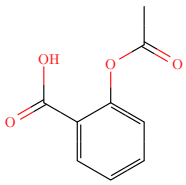
salicylic-acid



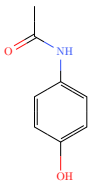
caffeine



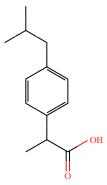
aspirin



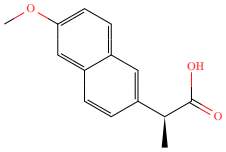
paracetamol



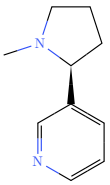
ibuprofen



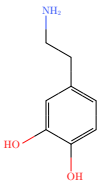
naproxen



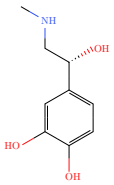
nicotine



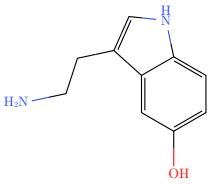
dopamine



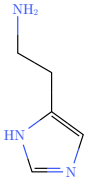
adrenaline



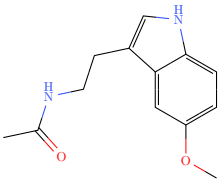
serotonin



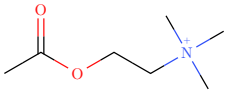
histamine



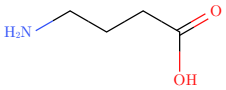
melatonin



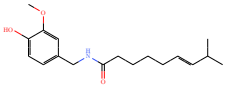
acetylcholine



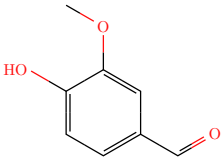
gamma-aminobutyric-acid



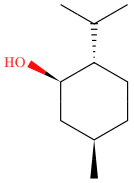
capsaicin



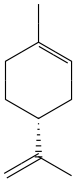
vanillin



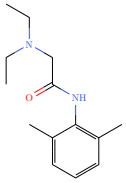
menthol



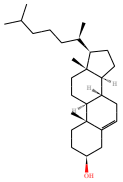
limonene



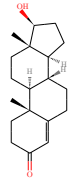
lidocaine



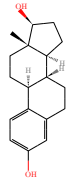
cholesterol



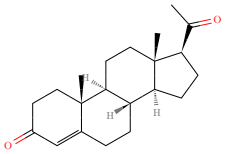
testosterone



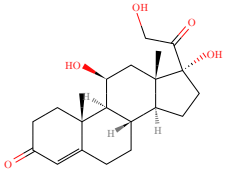
estradiol



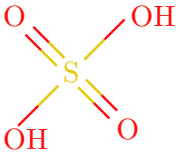
progesterone



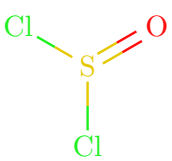
cortisol



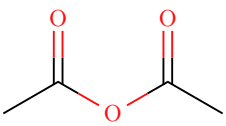
sulfuric-acid



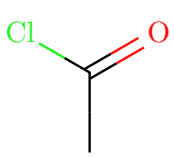
thionyl-chloride



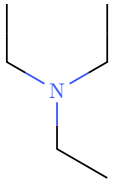
acetic-anhydride



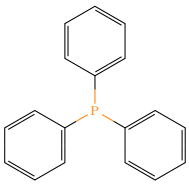
acetyl-chloride



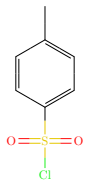
triethylamine



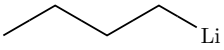
triphenylphosphine



tosyl-chloride



butyllithium



sodium-borohydride

Na^+ BH_4^-

lithium-aluminium-hydride

Li^+ AlH_4^-

sodium-chloride

Na^+ Cl^-

sodium-hydroxide

Na^+ OH^-

21 Quick reference

21.1 Syntax

Syntax	Meaning
`c n o …`	Lowercase aromatic atom notation.
`.`	Disconnected fragment (no bond): salts, hydrates.
`[...]`	Bracket atom (any element, charges, explicit H).
`[CH3:7]`	Atom map 7 on a bracket atom; shown with `show-maps: true`.
`@` / `@@`	Tetrahedral anticlockwise / clockwise.
`@SP1`–`@SP3`	Square planar (U / 4 / Z shape), depicted exactly.
`@TB1–20 @OH1–30 @AL1–2`	Extended stereo: not drawn; an error unless `undepicted-stereo: "omit"`.
`$`	Quadruple bond (four parallel lines).
`/ \`	Double-bond cis/trans geometry.
`!c`	Repeat the preceding acyclic-chain turn instead of alternating the zigzag.
`!w` / `!h`	Manual solid / hashed wedge (typed-smiles extension).
`!s` / `!d`	Wavy / dashed bond (typed-smiles extension).
`{label}`	Abbreviated group pseudo-atom.
`{>label}`	Abbreviated group anchored at the glyph after `>`.
`{label|style}`	Colored abbreviated group.
`{label|lp=N}`	Declare N (1–4) lone pairs on a label (style optional before it).
`{label|offset=(x,y)}`	Move one custom-label atom and its incident bond endpoints in page-space bond-length units.

21.2 `smiles()` options

Option	Default	Effect
`style`	`"default"`	Journal preset: `"acs"`, `"rsc"`, `"nature"`, `"wiley"`.
`scale`	`1.0`	Balanced size of everything.
`bond-length`	`none`	Bond length only (`1.0` is 30 pt).
`font-size`	`none`	Atom-label size only.
`font`	`auto`	Atom-label font from the journal preset, or “New Computer Modern”.
`bond-stroke`	`none`	Bond width only.
`color`	`auto`	CPK colors for `"default"`; monochrome for journal presets.
`fg`	`auto`	Foreground for bonds/carbon labels; `auto` inherits the text color.
`theme`	`auto`	CPK palette variant; `auto` goes dark when `fg` is light.

`rotation`	`0deg`	Rotate, labels stay upright.
`mirror`	`none`	Optional `"horizontal"` or `"vertical"` reflection.
`show-h`	`()`	Label selected implicit hydrogens; use `"all"` for every atom or `"skeleton"` for separate H atoms and bonds.
`aromatic`	`"kekule"`	Lowercase-aromatic rings as doubles or `"circle"`.
`charge-style`	`"plain"`	Formal charges as a bare sign or `"circled"` signs.
`atom-annotations`	`()`	Small gray side labels as `(index, content)` or `(index, content, offset)` tuples.
`opacity`	`100%`	Fade the whole drawing (ghost molecules).
`bond-customizations`	`()`	Per-bond `color`, `stroke`, `opacity` keyed by `bond(i, j)`.
`lone-pairs`	`none`	Draw lone pairs as `"dots"` or `"lines"`.
`atom-colors`	`(:)`	Color overrides: `O: red` for elements, `"{PPh3}": blue` for labels.
`show-indices`	`false`	Stamp atom indices for writing references.
`abbreviate`	`none`	Catalogue group names, or `"all"`, to draw as labels.
`show-maps`	`false`	Label mapped atoms such as `[CH3:7]` with `:7`.
`undepicted-stereo`	`"error"`	Stereochemistry the drawing cannot show: report it, or `"omit"` it.
`highlight-smarts`	`()`	SMARTS string, `(pattern:, include-atoms:, include-hydrogens:)` dictionary, or tuple mixing both.
`highlight-groups`	`()`	Group name, `(group:, include-atoms:, include-hydrogens:)` dictionary, or tuple mixing both.
`highlight-colors`	`auto`	Non-empty color tuple; cycle across matches.
`highlight-unmatched`	`"error"`	Diagnostic on absent matches; `"ignore"` permits absence.
`highlight-overlap`	`"merge"`	`"merge"` paints each translucent color as one shape; `"stack"` paints every piece separately.
`..annotations`	—	`arrow()` / `highlight()` items on this molecule.

21.3 `reaction()` options

Option	Default	Effect
`gap-h`	`1.5em`	Horizontal gap between items.
`gap-v`	`1.5em`	Vertical gap between rows.
`scale`	`1.0`	Uniform scale applied to the whole scheme.
`breakable`	`false`	Allow the scheme to split across pages.
`flow`	`"right"`	Writing direction: `"right"`, `"left"`, `"up"`, or `"down"` (left/up reflect the scheme).
`show-indices`	`false`	Default atom-index overlay for string SMILES molecules in this reaction.

`fit`	`none`	`"width"` shrinks the whole scheme uniformly to the available width after `scale`, never enlarging it.
`highlight-overlap`	`"merge"`	Default highlight painting for this reaction: `"merge"` or `"stack"`. A `mol()` can set its own value.

21.4 `rxn-arrow()` options

Option	Default	Effect
`above`	`none`	Condition label (content or `mol()`) above a horizontal arrow, or right of a vertical arrow.
`below`	`none`	Condition label (content or `mol()`) below a horizontal arrow, or left of a vertical arrow.
`dir`	`auto`	Arrow direction: `"right"`, `"left"`, `"up"`, `"down"`, or `auto` (follows `reaction(flow:)`).
`kind`	`"single"`	Arrow style: `"single"`, `"equilibrium"`, `"equilibrium-filled"`, `"dashed"`, or `"wavy"`.
`scale`	`1.0`	Uniform arrow scale, including condition labels.
`color`	`auto`	Arrow color; `auto` inherits the surrounding text color.
`stroke`	`auto`	Shaft width before scaling; `auto` matches a default 0.9pt molecule bond.
`label-size`	`auto`	Condition-label text size before scaling; `auto` is `0.8em` of the surrounding text.

21.5 Data helpers

Helper	Purpose
`ce(chem, font:, font-size:, charge-style:, stacked-charges:, ..args)`	Chemical formula or equation through chemformula; `stacked-charges: false` writes charges after subscripts and `charge-style: "circled"` circles the sign.
`mol-formula(smiles)`	Hill-ordered molecular formula content; errors on wildcards, abbreviations, and isotopes.
`mol-weight(smiles)`	Molecular weight in g/mol as a `float`; errors on wildcards, abbreviations, and isotopes.
`smiles-inline(smiles, height:, baseline:, ..args)`	Molecule scaled and baseline-aligned for running text.
`smiles-cetz(smiles, name:, origin:, fg:, theme:, ..opts)`	Molecule as CeTZ elements with `atom-<i>` / `bond-<i>-<j>` / `center` anchors.

`molecules.<name>`	Bundled SMILES string for a common molecule (Section 20); `molecules.at("name")` for computed names.
`substructure-matches(smiles, pattern)`	Array of `(atoms:, bonds:)` index dictionaries for every SMARTS match; `()` when absent.
`functional-groups.<name>`	SMARTS definition behind each `highlight-groups` name.

21.6 Mechanism helpers

Helper	Purpose
`mol(spec, ..annotations, label:, offset:, ..opts)`	Reaction item; string or aligned `spec` has addressable atoms; positional arrows/highlights use local references; `offset` nudges in page coordinates; per-molecule `scale` also works in mechanism mode.
`cycle(..items, radius:, start:, clockwise:, scale:, reagent-bend:, arc-gap:)`	Catalytic cycle: species on a ring with arc arrows.
`step(label:, into:, out:, bend:, merge:, rotation:, *-offset:)`	A cycle arc: transformation label, reagent in/out, side-arrow bend, tangential merge, label rotation, and per-piece offsets.
`atom / bond / lp / species`	Atom-index references (optional `offset:`).
`arrow(from:, to:, label:, color:, stroke:, bend:, angle:, half:, heads:, head-length:, head-width:, style:)`	Curly electron-pushing arrow (default `color: black`); `stroke: auto` matches molecule bonds and scales with the drawing; `head-length` / `head-width` size the tip; `heads` is `"end"` / `"both"` / `"none"`, `style` is `"solid"` / `"dashed"` / `"wavy"`.
`highlight(ref, fill:, stroke:, radius:, include-atoms:, include-hydrogens:)`	Shade one atom/bond reference or an array of references, and the displayed H of selected heteroatoms.

`brackets(body, sup:, sub:)`	Square brackets around content; inside `reaction()`, accepts reference-transparent reaction items too.

21.7 Molecule series

Helper	Purpose
`align-molecules(molecules, scaffold:, atoms:, reference:, rotation:, mirror:, allow-reflection:)`	Array of aligned molecules whose shared scaffold keeps the reference's orientation; fields `smiles`, `rotation`, `mirror`, `atoms`, `deviation`.
`molecule-grid(columns:, scale:, bond-length:, sizing:, scaffold:, column-gutter:, row-gutter:, label-gap:, breakable:, ..items)`	Compound grid at one shared bond length with row-aligned captions.

21.8 Label color names

Named styles for `{label|style}` — any of these plus any `#RRGGBB` hex:

Name	Swatch	Name	Swatch	Name	Swatch
red		orange		yellow	
brown		green		lime	
teal		cyan		blue	
navy		purple		pink	
black		gray		silver	
maroon		white			